Supermemory AI

Complete Documentation

Memory API for the AI Era

Source: https://supermemory.ai/docs

Generated: April 2026

Pages: 103 documentation pages

Table of Contents

Getting Started 7

add-memories . . . 7

authentication . . . 8

document-operations . . . 10

intro . . . 11

memory-operations . . . 12

search . . . 14

user-profiles . . . 15

vibe-coding . . . 17

Core Concepts 18

concepts content-types . . . 19

concepts customization . . . 20

concepts filtering . . . 22

concepts graph-memory . . . 23

concepts how-it-works . . . 24

concepts memory-vs-rag . . . 26

concepts super-rag . . . 27

concepts user-profiles . . . 28

API Reference - Connections 30

api-reference connections configure-connection . . . 30

api-reference connections create-connection . . . 31

api-reference connections delete-connection-by-id . . . 32

api-reference connections delete-connection . . . 34

api-reference connections fetch-resources . . . 35

api-reference connections get-connection-by-id . . . 36

api-reference connections get-connection-by-provider . . . 38

api-reference connections list-connections 39
api-reference connections list-documents 40
api-reference connections sync-connection 42
API Reference - Content Management 43
api-reference content-management create-memories-directly 43
api-reference content-management forget-a-memory 45
api-reference content-management list-memory-entries-with-history 46
api-reference content-management update-a-memory-creates-new-version 47
API Reference - Documents 49
api-reference documents get-document-chunks 49
api-reference documents get-document 50
api-reference documents get-processing-documents 52
api-reference documents list-documents 53
api-reference documents search-documents 54
api-reference ingest batch-add-documents 55
api-reference ingest bulk-delete-documents 57
API Reference - Ingest 58
api-reference ingest add-document 58
api-reference ingest delete-document-by-id-or-customid 60
api-reference ingest ingest-or-update-conversation 61
api-reference ingest update-document 62
api-reference ingest upload-a-file 64
API Reference - Knowledge Graph 65
api-reference knowledge-graph get-graph-bounds 65
api-reference knowledge-graph get-graph-statistics 67
api-reference knowledge-graph get-graph-viewport-data 68
API Reference - Profiles 70
api-reference profiles get-user-profile 70

API Reference - Recall Search 71

api-reference recall-search search-memory-entries . . . 71

API Reference - Settings 73

api-reference settings get-settings . . . 73

api-reference settings reset-organization-data . . . 74

api-reference settings toggle-overage . . . 75

api-reference settings update-settings . . . 77

api-reference spaces get-container-tag-settings . . . 78

api-reference spaces update-container-tag-settings . . . 79

API Reference - Spaces 81

api-reference spaces delete-container-tag . . . 81

api-reference spaces merge-container-tags . . . 82

Connectors 84

connectors github . . . 84

connectors gmail . . . 85

connectors google-drive . . . 87

connectors notion . . . 88

connectors onedrive . . . 89

connectors overview . . . 91

connectors s3 . . . 92

connectors troubleshooting . . . 93

connectors web-crawler . . . 95

memory-api connectors managing-resources . . . 96

Integrations 97

integrations agent-framework . . . 97

integrations agno . . . 99

integrations ai-sdk . . . 100

integrations claude-code . . . 101

integrations claude-memory . . . 103
integrations crewai . . . 104
integrations hermes . . . 105
integrations langchain . . . 106
integrations langgraph . . . 108
integrations mastra . . . 109
integrations memory-graph . . . 110
integrations n8n . . . 112
integrations openai-agents-sdk . . . 113
integrations openai . . . 114
integrations openclaw . . . 116
integrations opencode . . . 117
integrations pipecat . . . 118
integrations supermemory-sdk . . . 120
integrations viasocket . . . 121
integrations voltagent . . . 123
integrations zapier . . . 124

Cookbook & Examples 126

cookbook ai-sdk-integration . . . 126
Building an AI Compliance Chatbot With Supermemory and Google Drive . . . 127
{selectedCustomer.name} . . . 141
cookbook document-qa . . . 142
cookbook overview . . . 143
Build Your Own Perplexity in 15 Minutes With Supermemory . . . 145
 Personal AI Assistant . . . 156

Model Context Protocol (MCP) 157

supermemory-mcp mcp . . . 157
supermemory-mcp setup . . . 159

MemoryBench 160

memorybench architecture 160

memorybench cli 162

memorybench contributing 163

memorybench extend-benchmark 165

memorybench extend-provider 166

Search code, repositories, users, issues, pull requests... 168

Migration 180

migration from-zep 180

Changelog 181

changelog overview 181

Getting Started

Pages	Content Size
8	60,130 characters

add-memories

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Ingesting context to supermemory

> Add text, files, and URLs to Supermemory

Send any raw content to Supermemory — conversations, documents, files, URLs. We extract the memories
automatically.


**Use `customId`** to identify your content (conversation ID, document ID, etc.). This enables updates and
prevents duplicates.


## Quick Start



```typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory();

// Add text content
await client.add({
content: "Machine learning enables computers to learn from data",
containerTag: "user_123",
metadata: { category: "ai" }
});

// Add a URL (auto-extracted)
await client.add({
content: "https://youtube.com/watch?v=dQw4w9WgXcQ",
containerTag: "user_123"
});
```



```python theme={null}
from supermemory import Supermemory
```
```

```
client = Supermemory()

# Add text content
client.add(
content="Machine learning enables computers to learn from data",
container_tag="user_123",
metadata={"category": "ai"}
)

# Add a URL (auto-extracted)
client.add(
content="https://youtube.com/watch?v=dQw4w9WgXcQ",
container_tag="user_123"
)
```



```bash theme={null}
curl -X POST "https://api.supermemory.ai/v3/documents" \
-H "Authorization: Bearer $SUPERMEMORY_API_KEY" \
-H "Content-Type: application/json" \
-d '{
"content": "Machine learning enables computers to learn from data",
"containerTag": "user_123",
"metadata": {"category": "ai"}
}'
```


Response:

```json theme={null}
{ "id": "abc123", "status": "queued" }
```

***

Updating Content

Use `customId` to update existing documents or conversations. When
... (truncated)
```

# authentication

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

Authentication

> API keys, scoped keys, and connector branding.
```
```

API Keys

All API requests require authentication using a Bearer token. Get your API key from the [Developer Platform](https://console.supermemory.ai).

Include your key in all requests:

```bash cURL theme={null}
curl https://api.supermemory.ai/v3/search \
--header 'Authorization: Bearer YOUR_API_KEY' \
--header 'Content-Type: application/json' \
-d '{"q": "hello"}'
```

```typescript TypeScript theme={null}
import Supermemory from "supermemory";

const client = new Supermemory({ apiKey: "YOUR_API_KEY" });
```

```python Python theme={null}
from supermemory import Supermemory

client = Supermemory(api_key="YOUR_API_KEY")
```

***

Connector Branding

When users connect external services (Google Drive, Notion, OneDrive), they see a "Log in to **Supermemory**" prompt by default. You can replace this with your own app name by providing your own OAuth credentials via the settings endpoint.

```typescript theme={null}
await client.settings.update({
googleDriveCustomKeyEnabled: true,
googleDriveClientId: "your-client-id.apps.googleusercontent.com",
googleDriveClientSecret: "your-client-secret"
});
```

This works for Google Drive, Notion, and OneDrive. See the full setup in [Customization](/concepts/customization).

***

Scoped API Keys

Scoped keys are restricted to a single `containerTag`. They can only access documents and search within that container — useful for giving limited access to specific projects, users, or tenants without

```
exposing your full API key.

**Allowed endpoints:** `/v3/documents`, `/v3/memories`, `/v4/memories`, `/v3/search`, `/v4/search`,
`/v4/profile`

### Create a scoped key

```bash theme={null}
curl https://api.supermemory.ai/v3/auth/s
... (truncated)
```

document-operations

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Document Operations

> List, get, update, and delete your ingested documents

Manage documents after ingestion using the SDK.

## List Documents

Retrieve paginated documents with filtering.



```typescript theme={null}
const documents = await client.documents.list({
limit: 10,
containerTags: ["user_123"]
});

documents.memories.forEach(d => {
console.log(d.id, d.title, d.status);
});
```



```python theme={null}
documents = client.documents.list(
limit=10,
container_tags=["user_123"]
)

for doc in documents.memories:
print(doc.id, doc.title, doc.status)
```
```
```

```bash theme={null}
curl -X POST "https://api.supermemory.ai/v3/documents/list" \
-H "Authorization: Bearer $SUPERMEMORY_API_KEY" \
-H "Content-Type: application/json" \
-d '{"limit": 10, "containerTags": ["user_123"]}'
```


Response:

```json theme={null}
{
"memories": [
{
"id": "doc_abc123",
"title": "Meeting notes",
"status": "done",
"type": "text",
"createdAt": "2024-01-15T10:30:00Z",
"containerTags": ["user_123"],
"metadata": { "source": "slack" }
}
],
"pagination": {
"currentPage": 1,
"totalPages": 3,
"totalItems": 25
}
}
```

Parameters

Parameter	Type	Default	Description
`limit`	number	50	Items per page (max 200)
`page`	number	1	Page number
`containerTags`	string\[]	—	Filter by tags
`sort`	string	`createdAt`	Sort by `createdAt` or `u
... (truncated)

intro

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Overview — What is Supermemory?

Supermemory is the long-term and short-term memory and context infrastructure for AI agents. It is the
[state of the art](https://supermemory.ai/research) across multiple different benchmarks, like
LongMemEval and LoCoMo.
```

```
With supermemory, developers can provide perfect recall about their users to build AI agents that are more
intelligent, more personalized, and more consistent. Additionally, *supermemory* has all the pieces of
the context stack built in:

* [Agent memory](/concepts/graph-memory)
* [Content extraction](/concepts/content-types)
* [Connectors and syncing](/connectors/overview)
* [Managed RAG platform](/concepts/super-rag)

All this, coming together, makes supermemory the best abstraction to provide to agents.

## How does it work? (at a glance)



* You send Supermemory text, files, and chats.
* Supermemory [intelligently indexes them](/concepts/how-it-works) and builds a semantic understanding
graph on top of an entity (e.g., a user, a document, a project, an organization).
* At query time, we fetch only the most relevant context and pass it to your models.

## Supermemory is context engineering.

#### Ingestion and Extraction

Supermemory handles all the extraction, for [any data type that you have](/concepts/content-types).

* Text
* Conversations
* Files (PDF, Images, Docs)
* Even videos!

... and then,

We offer three ways to add context to your LLMs:

#### Memory API — Learned user context



Supermemory learns and builds the memory for the user. These are extracted facts about the user, that:

* [Evolve on top of existing context about the user](/concepts/graph-memory), **in real time**
* Handle **knowledge updates, temporal changes, forgetfulness**
* Creates a **user profile** as the default context provider for the LLM.

*This c
... (truncated)
```

memory-operations

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
```

> Use this file to discover all available pages before exploring further.

Memory Operations

> Advanced memory operations (v4 API)

These v4 endpoints operate on extracted memories (not raw documents). SDK support coming soon — use fetch or cURL for now.

For document management (list, get, update, delete), see [Document Operations](/document-operations).
For ingesting raw content (text, files, URLs) through the processing pipeline, see [Add Context](/add-memories).

Create Memories

Create memories directly without going through the document ingestion workflow. Memories are embedded and immediately searchable.

This is useful for storing user preferences, traits, or any structured facts where you already know the exact memory content.

```typescript theme={null}
const response = await fetch("https://api.supermemory.ai/v4/memories", {
method: "POST",
headers: {
"Authorization": `Bearer ${API_KEY}`,
"Content-Type": "application/json"
},
body: JSON.stringify({
memories: [
{
content: "John prefers dark mode",
isStatic: false,
metadata: { source: "user_preference" }
},
{
content: "John is from Seattle",
isStatic: true
}
],
containerTag: "user_123"
})
});

const data = await response.json();
// {
// documentId: "abc123",
// memories: [
// { id: "mem_1", memory: "John prefers dark mode", isStatic: false, createdAt: "2025-..." },
// { id: "mem_2", memory: "John is from Seattle", isStatic: true, createdAt: "2025-..." }
// ]
// }
```

```bash theme={null}
curl -X POST "https://api.supermemory.ai/v4/memories" \
-H "Authorization: Bearer $SUPERMEMORY_API_KEY" \
-H "Content-Type: application/json" \

... (truncated)
```

search

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Search

> Semantic search across your memories and documents

Search through your memories and documents with a single API call.


**Use `searchMode: "hybrid"`** for best results. It searches both memories and document chunks, returning
the most relevant content.


## Quick Start



```typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory();

const results = await client.search.memories({
q: "machine learning",
containerTag: "user_123",
searchMode: "hybrid",
limit: 5
});

results.results.forEach(result => {
console.log(result.memory || result.chunk, result.similarity);
});
```



```python theme={null}
from supermemory import Supermemory
```
```

```
client = Supermemory()

results = client.search.memories(
q="machine learning",
container_tag="user_123",
search_mode="hybrid",
limit=5
)

for result in results.results:
print(result.memory or result.chunk, result.similarity)
```



```bash theme={null}
curl -X POST "https://api.supermemory.ai/v4/search" \
-H "Authorization: Bearer $SUPERMEMORY_API_KEY" \
-H "Content-Type: application/json" \
-d '{
"q": "machine learning",
"containerTag": "user_123",
"searchMode": "hybrid",
"limit": 5
}'
```


Response:

```json theme={null}
{
"results": [
{
"id": "mem_xyz",
"memory": "User is interested in machine learning for product recommendations",
"similarity": 0.91,
"metadata": { "topic": "interests" },
"updatedAt": "2024-01-15T10:30:00.000Z",
"version": 1
},
{
"id": "chunk_abc",
"chunk": "Machine learning enables personalized experiences at scale
... (truncated)
```

user-profiles

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.
```

User Profiles

> Fetch and use automatically maintained user context

User profiles are extremely short summaries of context about an entity (Usually a user, but can be anything) which includes both the *static* facts about them, as well as a few recent episodes.

> You can think of these as a dynamic compaction that's done by supermemory in real-time.

This profile should be injected into the agent context for truly personalized experiences. To read more, visit [User profiles - Concept](/concepts/user-profiles)

Get a user's profile — their static facts and dynamic context — with a single API call.

Profiles are built automatically as you [ingest content](/add-memories). No setup required.

Quick Start

```typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory();

const { profile } = await client.profile({
containerTag: "user_123"
});

console.log(profile.static); // Long-term facts
console.log(profile.dynamic); // Recent context
```

```python theme={null}
from supermemory import Supermemory

client = Supermemory()

result = client.profile(container_tag="user_123")

print(result.profile.static) # Long-term facts
print(result.profile.dynamic) # Recent context
```

```bash theme={null}
curl -X POST "https://api.supermemory.ai/v4/profile" \
-H "Authorization: Bearer $SUPERMEMORY_API_KEY" \
-H "Content-Type: application/json" \
-d '{"containerTag": "user_123"}'
```

```
**Response:**

```json theme={null}
{
"profile": {
"static": [
"User is a software engineer",
"User specializes in Python and React",
"User prefers dark mode interfaces"
],
"dynamic": [
"User is working on Project Alpha",

... (truncated)
```

vibe-coding

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Vibe Coding Setup

> Automatic Supermemory integration using AI coding agents

Get your AI coding agent to integrate Supermemory in minutes. Copy the prompt below, paste it into
Claude/GPT/Cursor, and let it do the work.

## Quick Setup



Give your agent a way to reference and search through supermemory docs.



Paste one prompt, answer questions, get working code



Interactive guided setup



## MCP Server

Give your agent a way to reference and search through supermemory docs.

### Quick Install

```bash theme={null}
```

```
npx -y install-mcp@latest https://supermemory.ai/docs/mcp --client claude-code --oauth=no -y
```

Replace `claude` with: `cursor`, `opencode`, or `vscode`

***

The Prompt

Copy everything in the code block below and paste it into your AI coding agent. It will ask you questions and generate complete integration code.

After adding the MCP, paste this in your agent session:

```
You are integrating Supermemory into my application. Supermemory provides user memory, semantic search, and automatic knowledge extraction for AI applications.

Note: You can always reference the documentation by using the **SearchSupermemoryDocs MCP** or running a web search tool for content on **supermemory.ai/docs**.

STEP 1: ASK ME THESE QUESTIONS

1. What are you building?
- Personal chatbot/assistant
- Team knowledge base
- Customer support bot
- Document Q&A
- Other

2. How do you want to integrate?
- Vercel AI SDK (@supermemory/tools)
- OpenAI plugins
- Direct SDK (supermemory npm/pip)
- Direct API calls

3. Data model?
- Individual users only → containerTag: userId
- Organizations only → containerTag: orgId
- Both users AND orgs → ask for strategy

4. Do you want USER PROFILE
... (truncated)
```

Core Concepts

Pages	Content Size

8	49,390 characters

concepts content-types

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Supported Content Types

> All the content formats Supermemory can ingest and process

Supermemory automatically extracts and indexes content from various formats. Just send it—we handle the
rest. See [Add Memories](/add-memories) to learn how to ingest content via the API.

## Text Content

Raw text, conversations, notes, or any string content.

```typescript theme={null}
await client.add({
content: "User prefers dark mode and uses vim keybindings",
containerTags: ["user_123"]
});
```

Best for: Chat messages, user preferences, notes, logs, transcripts.

***

URLs & Web Pages

Send a URL and Supermemory fetches, extracts, and indexes the content.

```typescript theme={null}
await client.add({
content: "https://docs.example.com/api-reference",
containerTags: ["documentation"]
});
```

Extracts: Article text, headings, metadata. Strips navigation, ads, boilerplate.

***

Documents

PDF

```typescript theme={null}
await client.add({
content: pdfBase64,
contentType: "pdf",
title: "Q4 Financial Report"
```
```

```
});
```

Extracts: Text, tables, headers. OCR for scanned documents.

Microsoft Office

Format	Extension	Content Type
Word	`.docx`	`docx`
Excel	`.xlsx`	`xlsx`
PowerPoint	`.pptx`	`pptx`

```typescript theme={null}
await client.add({
content: docxBase64,
contentType: "docx",
title: "Product Roadmap"
});
```

Google Workspace

Automatically handled via [Google Drive connector](/connectors/google-drive):

* Google Docs
* Google Sheets
* Google Slides

***

Code & Markdown

```typescript theme={null}
// Markdown
await client.add({
content: markdownContent,
contentType: "md",
title: "README.md"
});

// Code files (auto-detected
... (truncated)
```

concepts customization

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Customizing for Your Use Case

> Configure Supermemory's behavior for your specific application
```

Configure how Supermemory processes and retrieves content for your specific use case.

Filter Prompts

Tell Supermemory what content matters during ingestion. This helps filter and prioritize what gets indexed.

```typescript theme={null}
// Example: Brand guidelines assistant
await client.settings.update({
shouldLLMFilter: true,
filterPrompt: `You are ingesting content for Brand.ai's brand guidelines system.

Index:
- Official brand values and mission statements
- Approved tone of voice guidelines
- Logo usage and visual identity docs
- Approved messaging and taglines

Skip:
- Draft documents and work-in-progress
- Outdated brand materials (pre-2024)
- Internal discussions about brand changes
- Competitor analysis docs`
});
```

```typescript theme={null}
filterPrompt: `Personal AI assistant. Prioritize recent content, action items,
and personal context. Exclude spam and duplicates.`
```

```typescript theme={null}
filterPrompt: `Customer support agent. Prioritize verified solutions, official docs,
and resolved tickets. Exclude internal discussions and PII.`
```

```typescript theme={null}
filterPrompt: `Legal research assistant. Prioritize precedents, current regulations,
and approved contract language. Exclude privileged communications.`
```

```typescript theme={null}
filterPrompt: `Financial analysis assistant. Prioritize latest reports, verified data,
and regulatory filings. Exclude speculative data and MNPI.`
```

```typescript theme={null}
filterPrompt: `Healthcare information assistant. Prioritize evidence-based guidelines
and FDA-ap
... (truncated)
```

concepts filtering

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Organizing & Filtering Memories

> Use container tags and metadata to organize and retrieve memories

Supermemory provides two ways to organize your memories:



**Organize memories** into isolated spaces by user, project, or workspace



**Query memories** by custom properties like category, status, or date



Both can be used independently or together for precise filtering.

***

## Container Tags

Container tags create isolated memory spaces. Use them to separate memories by user, project, or any
logical boundary.

### Adding Memories with Tags

```typescript theme={null}
await client.add({
content: "Meeting notes from Q1 planning",
containerTags: ["user_123"]
});
```

Searching with Tags

```typescript theme={null}
const results = await client.search.documents({
q: "planning notes",
containerTags: ["user_123"]
});
```
```

```
```

Container tags use **exact array matching**. A memory tagged `["user_123", "project_a"]` won't match a search for just `["user_123"]`.

Recommended Patterns

Pattern	Example	Use Case
User isolation	`user_{userId}`	Per-user memories
Project grouping	`project_{projectId}`	Project-specific content
Hierarchical	`org_{orgId}_team_{teamId}`	Multi-level organization

```typescript theme={null}
// Multi-tenant SaaS - isolate by organization and user
await client.add({
content: "Company policy document",
containerTags: ["org_acme_user_john"]
});

// Search only within that user's org context
const results = await client.search.documents({
q: "vacation policy",
containerTags: ["org_acme_user_john"]
});

... (truncated)
```

concepts graph-memory

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# How Graph Memory Works

> Automatic memory evolution, knowledge updates, and intelligent forgetting

Supermemory builds a living knowledge graph where memories connect to other memories. Unlike traditional knowledge graphs with entity-relation-entity triples, Supermemory's graph is **facts built on top of other facts**.

## Memory Relationships

When you add content, Supermemory extracts facts and automatically connects them to existing memories through three relationship types:

### Updates: Information Changes
```

```
When new information contradicts existing knowledge:

```
Memory 1: "Alex works at Google as a software engineer"
Memory 2: "Alex just started at Stripe as a PM"
↓
Memory 2 UPDATES Memory 1
```

The system tracks which memory is latest with `isLatest`, so searches return current information while preserving history.

Extends: Information Enriches

When new information adds detail without replacing:

```
Memory 1: "Alex works at Stripe as a PM"
Memory 2: "Alex focuses on payments infrastructure and leads a team of 5"
↓
Memory 2 EXTENDS Memory 1
```

Both memories remain valid—searches get richer context.

Derives: Information Infers

When Supermemory infers new facts from patterns:

```
Memory 1: "Alex is a PM at Stripe"
Memory 2: "Alex frequently discusses payment APIs and fraud detection"
↓
Derived: "Alex likely works on Stripe's core payments product"
```

These inferences surface insights you didn't explicitly state.

***

Automatic Memory Extraction

From a single conversation, Supermemory extracts multiple connected memories:

Input:

> "Had a great call with Alex. He's enjoying the new PM role at Stripe, though the
> payments infrastructure work is intense. He moved to Seattle for the job—got a
> place in Capitol Hill. Wants to g
... (truncated)
```

concepts how-it-works

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# How Supermemory Works

> Understanding the knowledge graph architecture that powers intelligent memory

Supermemory isn't just another document storage system. It's designed to mirror how human memory actually
works - forming connections, evolving over time, and generating insights from accumulated knowledge.



## The Mental Model

Traditional systems store files. Supermemory creates a living knowledge graph.



* Static files in folders
* No connections between content
* Search matches keywords
* Information stays frozen



* Dynamic knowledge graph
* Rich relationships between memories
* Semantic understanding
* Information evolves and connects



## Documents vs Memories

Understanding this distinction is crucial to using Supermemory effectively.

### Documents: Your Raw Input

Documents are what you provide - the raw materials:

* PDF files you upload
* Web pages you save
* Text you paste
* Images with text
* Videos to transcribe

Think of documents as books you hand to Supermemory. See [Content Types](/concepts/content-types) for the
full list of supported formats.

### Memories: Intelligent Knowledge Units

Memories are what Supermemory creates - the understanding:

* Semantic chunks with meaning
* Embedded for similarity search
* Connected through relationships
```

```
* Dynamically updated over time

Think of memories as the insights and connections your brain makes after reading those books.


**Key Insight**: When you upload a 50-page PDF, Supermemory doesn't just store it. It breaks it into
hundreds of interconnected memories, each understanding its context and relationships to your other
knowledge.


## Memory Relationships



The graph connects memories through three types of relationships. For a deeper dive into how these
relationships w
... (truncated)
```

concepts memory-vs-rag

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Memory vs RAG: Understanding the Difference

> Learn why agent memory and RAG are fundamentally different, and when to use each approach

Most developers confuse RAG (Retrieval-Augmented Generation) with agent memory. They're not the same
thing, and using RAG for memory is why your agents keep forgetting important context. Let's understand the
fundamental difference.

## The Core Problem

When building AI agents, developers often treat memory as just another retrieval problem. They store
conversations in a vector database, embed queries, and hope semantic search will surface the right
context.

**This approach fails because memory isn't about finding similar text—it's about understanding
relationships, temporal context, and user state over time.**

## Documents vs Memories in Supermemory

Supermemory makes a clear distinction between these two concepts:

### Documents: Raw Knowledge

Documents are the raw content you send to Supermemory—PDFs, web pages, text files. They represent static
knowledge that doesn't change based on who's accessing it.

**Characteristics:**

* **Stateless**: A document about Python programming is the same for everyone
```

```
* **Unversioned**: Content doesn't track changes over time
* **Universal**: Not linked to specific users or entities
* **Searchable**: Perfect for semantic similarity search

**Use Cases:**

* Company knowledge bases
* Technical documentation
* Research papers
* General reference material

### Memories: Contextual Understanding

Memories are the insights, preferences, and relationships extracted from documents and conversations.
They're tied to specific users or entities and evolve over time.

**Characteristics:**

* **Stateful**: "User prefers dark mode" is specific to that user
* **Temporal**: Tracks when facts became true or invalid
* **Person
... (truncated)
```

concepts super-rag

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# SuperRAG (Managed RAG as a service)

> Supermemory provides a managed RAG solution - extraction, indexing, storing, and retrieval.

Supermemory doesn't just store your content—it transforms it into optimized, searchable knowledge. Every
upload goes through an intelligent pipeline that extracts, chunks, and indexes content in the ideal way
for its type.

## Automatic Content Intelligence

When you add content, Supermemory:

1. **Detects the content type** — PDF, code, markdown, images, video, etc.
2. **Extracts content optimally** — Uses type-specific extraction (OCR for images, transcription for
audio)
3. **Chunks intelligently** — Applies the right chunking strategy for the content type
4. **Generates embeddings** — Creates vector representations for semantic search
5. **Builds relationships** — Connects new knowledge to existing memories

```typescript theme={null}
// Just add content — Supermemory handles the rest
await client.add({
content: pdfBase64,
contentType: "pdf",
```
```

```
title: "Technical Documentation"
});
```

No chunking strategies to configure. No embedding models to choose. It just works.

***

Smart Chunking by Content Type

Different content types need different chunking strategies. Supermemory applies the optimal approach automatically:

Documents (PDF, DOCX)

PDFs and documents are chunked by **semantic sections** — headers, paragraphs, and logical boundaries. This preserves context better than arbitrary character splits.

```
├── Executive Summary (chunk 1)
├── Introduction (chunk 2)
├── Section 1: Architecture
│ ├── Overview (chunk 3)
│ └── Components (chunk 4)
└── Conclusion (chunk 5)
```

Code

Code is chunked using [code-chunk](https://github.com/supermemoryai/code-chunk), our open-source library that understands AST (Abstract Syntax Tree) boundar
... (truncated)

concepts user-profiles

> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

User Profiles

> Automatically maintained context about your users

User profiles are **automatically maintained collections of facts about your users** that Supermemory builds from all their interactions. Think of it as a persistent "about me" document that's always up-to-date.

No search needed — comprehensive user info always ready

```
Profiles update as users interact with your system



## Why Profiles?

Traditional memory systems rely entirely on search:

| Problem | Search Only | With Profiles |
| ---------------- | ------------------------ | --------------- |
| Context retrieval | 3-5 queries | 1 call |
| Response time | 200-500ms | 50-100ms |
| Basic user info | Requires specific queries | Always available |

**Search is too narrow**: When you search for "project updates", you miss that the user prefers bullet
points, works in PST, and uses specific terminology.

**Profiles provide the foundation**: Instead of searching for basic context, profiles give your LLM a
complete picture of who the user is.

***

## Static vs Dynamic

Profiles separate two types of information:

### Static Profile

Long-term, stable facts:

* "Sarah is a senior software engineer at TechCorp"
* "Sarah specializes in distributed systems"
* "Sarah prefers technical docs over video tutorials"

### Dynamic Profile

Recent context and temporary states:

* "Sarah is migrating the payment service to microservices"
* "Sarah is preparing for a conference talk next month"
* "Sarah is debugging a memory leak in auth service"

***

## How It Works

Profiles are built automatically through ingestion:

1. **Ingest content** — Users [add documents](/add-memories), chat, or any content
2. **Extract facts** — AI analyze
... (truncated)
```

API Reference - Connections

Pages	Content Size
10	50,609 characters

api-reference connections configure-connection

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Configure connection

> Configure resources for a connection (supported providers: GitHub for now)



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v3/connections/{connectionId}/configure
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
```
```

```
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/connections/{connectionId}/configure:
post:
tags:
- Connections
summary: Configure connection
description: >-
Configure resources for a connection (supported providers: GitHub for
now)
operationId: postV3ConnectionsByConnectionIdConfigure
parameters:
- in: path
name: connectionId
schema:
type: string
required: true
requestBody:
required: true
co
... (truncated)
```

api-reference connections create-connection

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Create connection

> Initialize connection and get authorization URL



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v3/connections/{provider}
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
```
```

```
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/connections/{provider}:
post:
tags:
- Connections
summary: Create connection
description: Initialize connection and get authorization URL
operationId: postV3ConnectionsByProvider
parameters:
- in: path
name: provider
schema:
type: string
enum:
- notion
- google-drive
- onedrive
- gmail
- github
- web-crawler
- s3
requ
... (truncated)
```

api-reference connections delete-connection-by-id

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Delete connection by ID
```

> Delete a specific connection by ID

OpenAPI

```yaml https://api.supermemory.ai/v3/openapi delete /v3/connections/{connectionId}
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/connections/{connectionId}:
delete:
tags:
- Connections
summary: Delete connection by ID
description: Delete a specific connection by ID
operationId: deleteV3ConnectionsByConnectionId
parameters:
- in: path
name: connectionId
schema:
type: string
required: true
```

```
- in: query
name: deleteDocuments
schema:
default: 'true'
type: string
description: >-
Whether
... (truncated)
```

api-reference connections delete-connection

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Delete connection

> Delete connection for a specific provider and container tags



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi delete /v3/connections/{provider}
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
```
```

```
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/connections/{provider}:
delete:
tags:
- Connections
summary: Delete connection
description: Delete connection for a specific provider and container tags
operationId: deleteV3ConnectionsByProvider
parameters:
- in: path
name: provider
schema:
type: string
enum:
- notion
- google-drive
- onedrive
- gmail
- github
- web-crawler

... (truncated)
```

api-reference connections fetch-resources

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Fetch resources

> Fetch resources for a connection (supported providers: GitHub for now)



## OpenAPI

`´´´´yaml https://api.supermemory.ai/v3/openapi get /v3/connections/{connectionId}/resources
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
```

```
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/connections/{connectionId}/resources:
get:
tags:
- Connections
summary: Fetch resources
description: 'Fetch resources for a connection (supported providers: GitHub for now)'
operationId: getV3ConnectionsByConnectionIdResources
parameters:
- in: path
name: connectionId
schema:
type: string
required: true
- in: query
name: page
schema:
default: 1

... (truncated)
```

api-reference connections get-connection-by-id

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.
```

Get connection (by id)

> Get connection details with id

OpenAPI

```yaml https://api.supermemory.ai/v3/openapi get /v3/connections/{connectionId}
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/connections/{connectionId}:
get:
tags:
- Connections
summary: Get connection (by id)
description: Get connection details with id
operationId: getV3ConnectionsByConnectionId
parameters:
- in: path
name: connectionId
schema:
type: string
```

```
required: true
responses:
'200':
content:
application/json:
schema:
type: object
properties:
createdAt
... (truncated)
```

api-reference connections get-connection-by-provider

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Get connection (by provider)

> Get connection details with provider and container tags



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v3/connections/{provider}/connection
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
```
```

```
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/connections/{provider}/connection:
post:
tags:
- Connections
summary: Get connection (by provider)
description: Get connection details with provider and container tags
operationId: postV3ConnectionsByProviderConnection
parameters:
- in: path
name: provider
schema:
type: string
enum:
- notion
- google-drive
- onedrive
- gmail

... (truncated)
```

api-reference connections list-connections

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# List connections

> List all connections




## OpenAPI

`´´´´yaml https://api.supermemory.ai/v3/openapi post /v3/connections/list
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
```

```
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/connections/list:
post:
tags:
- Connections
summary: List connections
description: List all connections
operationId: postV3ConnectionsList
requestBody:
required: true
content:
application/json:
schema:
type: object
properties:
containerTags:
description: >-
Optional comma-separated list of container tags to filter
documents by
example:

... (truncated)
```

api-reference connections list-documents

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
```

> Use this file to discover all available pages before exploring further.

List documents

> List documents indexed for a provider and container tags

OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v3/connections/{provider}/documents
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/connections/{provider}/documents:
post:
tags:
- Connections
summary: List documents
description: List documents indexed for a provider and container tags
operationId: postV3ConnectionsByProviderDocuments
parameters:
- in: path
name: provider
```

```
schema:
type: string
enum:
- notion
- google-drive
- onedrive
- gmail
- github
- web-
... (truncated)
```

api-reference connections sync-connection

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Sync connection

> Initiate a manual sync of connections



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v3/connections/{provider}/import
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
```
```

```
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/connections/{provider}/import:
post:
tags:
- Connections
summary: Sync connection
description: Initiate a manual sync of connections
operationId: postV3ConnectionsByProviderImport
parameters:
- in: path
name: provider
schema:
type: string
enum:
- notion
- google-drive
- onedrive
- gmail
- github
- web-crawler
- s3
required
... (truncated)
```

API Reference - Content Management

Pages	Content Size
4	184,591 characters

api-reference content-management create-memories-directly

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Create memories directly

> Create memories directly, bypassing the document ingestion workflow. Generates embeddings and makes
them immediately searchable.
```

OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v4/memories
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v4/memories:
post:
tags:
- Content Management
summary: Create memories directly
description: >-
Create memories directly, bypassing the document ingestion workflow.
Generates embeddings and makes them immediately searchable.
operationId: postV4Memories
requestBody:
required: true
content:
application/json:
schema:
type: object
```

```
... (truncated)
```

api-reference content-management forget-a-memory

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Forget a memory

> Forget (soft delete) a memory entry. The memory is marked as forgotten but not permanently deleted.



## OpenAPI

`````yaml https://api.supermemory.ai/v3/openapi delete /v4/memories
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
```
```

```
paths:
/v4/memories:
delete:
tags:
- Content Management
summary: Forget a memory
description: >-
Forget (soft delete) a memory entry. The memory is marked as forgotten
but not permanently deleted.
operationId: deleteV4Memories
requestBody:
required: true
content:
application/json:
schema:
allOf:
- type: object
properties:
id:

... (truncated)
```

api-reference content-management list-memory-entries-with-history

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# List memory entries with history

> List all latest memory entries from specified container tags with their update history and source
documents



## OpenAPI

`````yaml https://api.supermemory.ai/v3/openapi post /v4/memories/list
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
```
```

```
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v4/memories/list:
post:
tags:
- Content Management
summary: List memory entries with history
description: >-
List all latest memory entries from specified container tags with their
update history and source documents
operationId: postV4MemoriesList
requestBody:
required: true
content:
application/json:
schema:
type: object
propertie
... (truncated)
```

api-reference content-management update-a-memory-creates-new-version

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Update a memory (creates new version)

> Update a memory by creating a new version. The original memory is preserved with isLatest=false.



## OpenAPI
```

```yaml https://api.supermemory.ai/v3/openapi patch /v4/memories
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v4/memories:
patch:
tags:
- Content Management
summary: Update a memory (creates new version)
description: >-
Update a memory by creating a new version. The original memory is
preserved with isLatest=false.
operationId: patchV4Memories
requestBody:
required: true
content:
application/json:
schema:
allOf:
- type: object
pro
... (truncated)
```

API Reference - Documents

Pages	Content Size
7	364,208 characters

api-reference documents get-document-chunks

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Get document chunks

> Get all chunks for a document, ordered by position



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi get /v3/documents/{id}/chunks
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
```
```

```
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/documents/{id}/chunks:
get:
tags:
- Documents
summary: Get document chunks
description: Get all chunks for a document, ordered by position
operationId: getV3DocumentsByIdChunks
parameters:
- in: path
name: id
schema:
type: string
required: true
responses:
'200':
content:
application/json:
schema:
type: object
properties:
doc
... (truncated)
```

api-reference documents get-document

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Get document

> Get a document by ID



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi get /v3/documents/{id}
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
```
```

```
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/documents/{id}:
get:
tags:
- Documents
summary: Get document
description: Get a document by ID
operationId: getV3DocumentsById
parameters:
- in: path
name: id
schema:
type: string
required: true
responses:
'200':
content:
application/json:
schema:
type: object
properties:
connectionId:
anyOf:
- anyOf:

... (truncated)
```

api-reference documents get-processing-documents

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Get processing documents

> Get documents that are currently being processed



## OpenAPI

`````yaml https://api.supermemory.ai/v3/openapi get /v3/documents/processing
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/documents/processing:
get:
tags:
- Documents
summary: Get processing documents
description: Get documents that are currently being processed
```
```

```
operationId: getV3DocumentsProcessing
responses:
'200':
content:
application/json:
schema:
type: object
properties:
documents:
type: array
items:
type: object

... (truncated)
```

api-reference documents list-documents

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# List documents

> Retrieves a paginated list of documents with their metadata and workflow status



## OpenAPI

`´´´´yaml https://api.supermemory.ai/v3/openapi post /v3/documents/list
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
```

```
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/documents/list:
post:
tags:
- Documents
summary: List documents
description: >-
Retrieves a paginated list of documents with their metadata and workflow
status
operationId: postV3DocumentsList
requestBody:
required: true
content:
application/json:
schema:
type: object
properties:
containerTags:
description: >-
Optional t
... (truncated)
```

api-reference documents search-documents

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Search documents

> Search memories with advanced filtering



## OpenAPI

`````yaml https://api.supermemory.ai/v3/openapi post /v3/search
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
```
```

```
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/search:
post:
tags:
- Documents
summary: Search documents
description: Search memories with advanced filtering
operationId: postV3Search
requestBody:
required: true
content:
application/json:
schema:
type: object
properties:
chunkThreshold:
description: >-
Threshold / sensitivity for chunk selection. 0 is least
sensitive (returns most chunks, more results), 1 is
... (truncated)
```

api-reference ingest batch-add-documents

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Batch add documents

> Add multiple documents in a single request. Each document can have any content type (text, url, file,
etc.) and metadata



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v3/documents/batch
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/documents/batch:
post:
tags:
- Ingest
summary: Batch add documents
description: >-
Add multiple documents in a single request. Each document can have any
```
```

```
content type (text, url, file, etc.) and metadata
operationId: postV3DocumentsBatch
requestBody:
required: true
content:
application/json:
schema:
type: object
properties:

... (truncated)
```

api-reference ingest bulk-delete-documents

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Bulk delete documents

> Bulk delete documents by IDs or container tags



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi delete /v3/documents/bulk
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
```
```

```
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/documents/bulk:
delete:
tags:
- Ingest
summary: Bulk delete documents
description: Bulk delete documents by IDs or container tags
operationId: deleteV3DocumentsBulk
requestBody:
required: true
content:
application/json:
schema:
type: object
properties:
ids:
description: Array of document IDs to delete (max 100 at once)
example:
- acxV5LHMEsG2
... (truncated)
```

API Reference - Ingest

Pages	Content Size
5	28,256 characters

api-reference ingest add-document

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Add document

> Add a document with any content type (text, url, file, etc.) and metadata
```

OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v3/documents
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/documents:
post:
tags:
- Ingest
summary: Add document
description: >-
Add a document with any content type (text, url, file, etc.) and
metadata
operationId: postV3Documents
requestBody:
required: true
content:
application/json:
schema:
type: object
required:
- content
properties:
```

```
content:
type: string
desc
... (truncated)
```

api-reference ingest delete-document-by-id-or-customid

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Delete document by ID or customId

> Delete a document by ID or customId



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi delete /v3/documents/{id}
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
```
```

```
- name: Documents
description: List, get, and search documents
paths:
/v3/documents/{id}:
delete:
tags:
- Ingest
summary: Delete document by ID or customId
description: Delete a document by ID or customId
operationId: deleteV3DocumentsById
parameters:
- in: path
name: id
schema:
type: string
required: true
responses:
'204':
description: Document deleted successfully
'401':
content:
application/json:
schema:
type: obje
... (truncated)
```

api-reference ingest ingest-or-update-conversation

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Ingest or update conversation

> Ingest or update a conversation



## OpenAPI

`´´´´yaml https://api.supermemory.ai/v3/openapi post /v4/conversations
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
```

```
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v4/conversations:
post:
tags:
- Ingest
summary: Ingest or update conversation
description: Ingest or update a conversation
operationId: postV4Conversations
requestBody:
required: true
content:
application/json:
schema:
type: object
properties:
conversationId:
type: string
minLength: 1
maxLength: 255
messages:
minItems: 1

... (truncated)
```

api-reference ingest update-document

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.
```

Update document

> Update a document with any content type (text, url, file, etc.) and metadata

OpenAPI

```yaml https://api.supermemory.ai/v3/openapi patch /v3/documents/{id}
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/documents/{id}:
patch:
tags:
- Ingest
summary: Update document
description: >-
Update a document with any content type (text, url, file, etc.) and
metadata
operationId: patchV3DocumentsById
parameters:
- in: path
name: id
```

```
schema:
type: string
required: true
requestBody:
required: true
content:
application/json:
schema:
type: object

... (truncated)
```

api-reference ingest upload-a-file

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Upload a file

> Upload a file to be processed



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v3/documents/file
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
```
```

```
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/documents/file:
post:
tags:
- Ingest
summary: Upload a file
description: Upload a file to be processed
operationId: postV3DocumentsFile
requestBody:
required: true
content:
multipart/form-data:
schema:
type: object
properties:
file:
description: File to upload and process
format: binary
type: string
containerTag:
description: >-

... (truncated)
```

API Reference - Knowledge Graph

Pages	Content Size
3	17,906 characters

api-reference knowledge-graph get-graph-bounds

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Get graph bounds

> Get the bounding box of all documents with spatial coordinates
```

OpenAPI

```yaml https://api.supermemory.ai/v3/openapi get /v3/graph/bounds
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/graph/bounds:
get:
tags:
- Knowledge Graph
summary: Get graph bounds
description: Get the bounding box of all documents with spatial coordinates
operationId: getV3GraphBounds
parameters:
- in: query
name: containerTags
schema:
type: string
maxLength: 10000
responses:
'200':
```

```
content:
application/json:
schema:
type: object
properties:

... (truncated)
```

api-reference knowledge-graph get-graph-statistics

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Get graph statistics

> Get summary statistics for the graph



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi get /v3/graph/stats
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
```
```

```
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/graph/stats:
get:
tags:
- Knowledge Graph
summary: Get graph statistics
description: Get summary statistics for the graph
operationId: getV3GraphStats
parameters:
- in: query
name: containerTags
schema:
type: string
maxLength: 10000
responses:
'200':
content:
application/json:
schema:
type: object
properties:
totalDocuments:

... (truncated)
```

api-reference knowledge-graph get-graph-viewport-data

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Get graph viewport data

> Fetch documents (with memories) and memory edges within a viewport



## OpenAPI

`´´´´yaml https://api.supermemory.ai/v3/openapi post /v3/graph/viewport
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
```

```
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/graph/viewport:
post:
tags:
- Knowledge Graph
summary: Get graph viewport data
description: Fetch documents (with memories) and memory edges within a viewport
operationId: postV3GraphViewport
requestBody:
required: true
content:
application/json:
schema:
type: object
properties:
viewport:
type: object
properties:
minX:

... (truncated)
```

API Reference - Profiles

Pages	Content Size
1	150,174 characters

api-reference profiles get-user-profile

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Get user profile

> Get user profile with optional search results



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v4/profile
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
```
```

```
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v4/profile:
post:
tags:
- Profiles
summary: Get user profile
description: Get user profile with optional search results
operationId: postV4Profile
requestBody:
required: true
content:
application/json:
schema:
type: object
properties:
q:
description: >-
Optional search query to include search results in the
response
type: string

... (truncated)
```

API Reference - Recall Search

Pages	Content Size
1	165,838 characters

api-reference recall-search search-memory-entries

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Search memory entries

> Search memory entries - Low latency for conversational



## OpenAPI

````yaml https://api.supermemory.ai/v3/openapi post /v4/search
```
```

```
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v4/search:
post:
tags:
- Recall (Search)
summary: Search memory entries
description: Search memory entries - Low latency for conversational
operationId: postV4Search
requestBody:
required: true
content:
application/json:
schema:
type: object
properties:
containerTag:
description: >-
Optional tag this search should be containerized by. This
can b
... (truncated)
```

API Reference - Settings

Pages	Content Size
6	38,894 characters

api-reference settings get-settings

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Get settings

> Get settings for an organization



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi get /v3/settings
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
```
```

```
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/settings:
get:
tags:
- Settings
summary: Get settings
description: Get settings for an organization
operationId: getV3Settings
responses:
'200':
content:
application/json:
schema:
type: object
properties:
excludeItems:
anyOf:
- anyOf:
- anyOf:
- type: string
- type: number

... (truncated)
```

api-reference settings reset-organization-data

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Reset organization data

> Reset organization content: removes documents, memories, spaces (except default project), connections,
and org settings. Preserves the org, members, and billing.



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v3/settings/reset
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
```
```

```
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/settings/reset:
post:
tags:
- Settings
summary: Reset organization data
description: >-
Reset organization content: removes documents, memories, spaces (except
default project), connections, and org settings. Preserves the org,
members, and billing.
operationId: postV3SettingsReset
requestBody:
required: true
content:

... (truncated)
```

api-reference settings toggle-overage

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Toggle overage

> Toggle overage billing for the organization
```

OpenAPI

```yaml https://api.supermemory.ai/v3/openapi patch /v3/settings/overage
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/settings/overage:
patch:
tags:
- Settings
summary: Toggle overage
description: Toggle overage billing for the organization
operationId: patchV3SettingsOverage
requestBody:
required: true
content:
application/json:
schema:
type: object
properties:
enabled:
type: boolean
```

```
required:
- enabled
responses:
'200':
content:
applicat
... (truncated)
```

api-reference settings update-settings

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Update settings

> Update settings for an organization



## OpenAPI

`´´´´yaml https://api.supermemory.ai/v3/openapi patch /v3/settings
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
```

```
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/settings:
patch:
tags:
- Settings
summary: Update settings
description: Update settings for an organization
operationId: patchV3Settings
requestBody:
required: true
content:
application/json:
schema:
type: object
properties:
excludeItems:
anyOf:
- anyOf:
- anyOf:
- type: string
- type: number

... (truncated)
```

api-reference spaces get-container-tag-settings

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Get container tag settings

> Get settings for a container tag




## OpenAPI

````yaml https://api.supermemory.ai/v3/openapi get /v3/container-tags/{containerTag}
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
```
```

```
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/container-tags/{containerTag}:
get:
tags:
- Spaces
summary: Get container tag settings
description: Get settings for a container tag
operationId: getV3ContainerTagsByContainerTag
parameters:
- schema:
type: string
in: path
name: containerTag
required: true
responses:
'200':
content:
application/json:
schema:
type: object
properties:

... (truncated)
```

api-reference spaces update-container-tag-settings

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Update container tag settings

> Update settings for a container tag



## OpenAPI

```yaml https://api.supermemory.ai/v3/openapi patch /v3/container-tags/{containerTag}
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/container-tags/{containerTag}:
patch:
tags:
- Spaces
summary: Update container tag settings
description: Update settings for a container tag
operationId: patchV3ContainerTagsByContainerTag
parameters:
- schema:
```
```

```
type: string
in: path
name: containerTag
required: true
requestBody:
required: true
content:
application/json:
schema:
type: object
proper
... (truncated)
```

API Reference - Spaces

Pages	Content Size
2	12,768 characters

api-reference spaces delete-container-tag

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Delete container tag

> Delete a container tag and all its documents and memories. Only organization owners and admins can
perform this action.



## OpenAPI

````yaml https://api.supermemory.ai/v3/openapi delete /v3/container-tags/{containerTag}
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
```
```

```
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/container-tags/{containerTag}:
delete:
tags:
- Spaces
summary: Delete container tag
description: >-
Delete a container tag and all its documents and memories. Only
organization owners and admins can perform this action.
operationId: deleteV3ContainerTagsByContainerTag
parameters:
- schema:
type: string
in: path
name: containerTag

... (truncated)
```

api-reference spaces merge-container-tags

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Merge container tags

> Merge multiple container tags into a target tag. All documents from the source tags will be updated to
reference the target tag, and the source tags will be deleted after successful merge.
```

OpenAPI

```yaml https://api.supermemory.ai/v3/openapi post /v3/container-tags/merge
openapi: 3.1.0
info:
title: supermemory API
description: The Memory API for the AI era
version: 3.0.0
servers:
- description: Production Server
url: https://api.supermemory.ai
security:
- bearerAuth: []
tags:
- name: Ingest
description: Ingest documents, files, URLs, conversations, and other content
- name: Recall (Search)
description: >-
Semantic recall across your content — supports memories, hybrid, and
documents modes
- name: Profiles
description: >-
Entity profiles for users, participants, or any entity — includes profile
search
- name: Content Management
description: List, get, update, and delete content and memories
- name: Spaces
description: Organize content into spaces (container tags)
- name: Knowledge Graph
description: Knowledge graph and entity relationships
- name: Connections
description: External service integrations
- name: Settings
description: Organization settings
- name: Analytics
description: Usage analytics and insights
- name: Documents
description: List, get, and search documents
paths:
/v3/container-tags/merge:
post:
tags:
- Spaces
summary: Merge container tags
description: >-
Merge multiple container tags into a target tag. All documents from the
source tags will be updated to reference the target tag, and the source
tags will be deleted after successful merge.
operationId: postV3ContainerTagsMerge

... (truncated)
```

Connectors

Pages	Content Size
10	122,971 characters

connectors github

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# GitHub Connector

> Connect GitHub repositories to sync documentation files into your Supermemory knowledge base

Connect GitHub repositories to sync documentation files into your Supermemory knowledge base with OAuth
authentication, webhook support, and automatic incremental syncing.


The GitHub connector requires a **Scale Plan** or **Enterprise Plan**.


## Quick Setup

### 1. Create GitHub Connection



```typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY!
});

const connection = await client.connections.create('github', {
redirectUrl: 'https://yourapp.com/auth/github/callback',
containerTags: ['user-123', 'github-sync'],
documentLimit: 5000,
metadata: {
source: 'github',
team: 'engineering'
}
});

// Redirect user to GitHub OAuth
window.location.href = connection.authLink;
console.log('Auth expires in:', connection.expiresIn);
```
```

```python theme={null}
from supermemory import Supermemory
import os

client = Supermemory(api_key=os.environ.get("SUPERMEMORY_API_KEY"))

connection = client.connections.create(
'github',
redirect_url='https://yourapp.com/auth/github/callback',
container_tags=['user-123', 'github-sync'],
document_limit=10000,
metadata={
'source': 'github',
'team': 'engineering'
}
)

# Redirect user to GitHub OAuth
print(f'Redirect to: {connection.auth_link}')
print(f'Expires in: {connection.expires_in}')
```



```bash theme={null}
curl -X POST "https://api.supermemory.ai/v3/connections/github" \
-H "Authorization: Bearer $SUPERMEMORY_API_KEY" \
-H
... (truncated)
```

connectors gmail

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Gmail Connector

> Sync email threads from Gmail with real-time Pub/Sub webhooks and incremental sync

Connect Gmail to automatically sync email threads into your supermemory knowledge base. Supports
real-time updates via Google Cloud Pub/Sub webhooks and incremental synchronization.


**Scale Plan Required:** The Gmail connector is available on Scale and Enterprise plans only.


## Quick Setup

### 1. Create Gmail Connection
```

```typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY!
});

const connection = await client.connections.create('gmail', {
redirectUrl: 'https://yourapp.com/auth/gmail/callback',
containerTags: ['user-123', 'gmail-sync'],
documentLimit: 5000,
metadata: {
source: 'gmail',
department: 'support'
}
});

// Redirect user to Google OAuth
window.location.href = connection.authLink;
console.log('Auth expires in:', connection.expiresIn);
```

```python theme={null}
from supermemory import Supermemory
import os

client = Supermemory(api_key=os.environ.get("SUPERMEMORY_API_KEY"))

connection = client.connections.create(
'gmail',
redirect_url='https://yourapp.com/auth/gmail/callback',
container_tags=['user-123', 'gmail-sync'],
document_limit=5000,
metadata={
'source': 'gmail',
'department': 'support'
}
)

# Redirect user to Google OAuth
print(f'Redirect to: {connection.auth_link}')
print(f'Expires in: {connection.expires_in}')
```

```bash theme={null}
curl -X POST "https://api.supermemory.ai/v3/connections/gmail" \
-H "Authorization: Bearer $SUPERMEMORY_API_K
... (truncated)
```

connectors google-drive

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Google Drive Connector

> Connect Google Drive to sync documents into your Supermemory knowledge base

Connect Google Drive to sync documents into your Supermemory knowledge base with OAuth authentication and
custom app support.

## Quick Setup

### 1. Create Google Drive Connection



```typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY!
});

const connection = await client.connections.create('google-drive', {
redirectUrl: 'https://yourapp.com/auth/google-drive/callback',
containerTags: ['user-123', 'gdrive-sync'],
documentLimit: 3000,
metadata: {
source: 'google-drive',
department: 'engineering'
}
});

// Redirect user to Google OAuth
window.location.href = connection.authLink;
console.log('Auth expires in:', connection.expiresIn);
```



```python theme={null}
from supermemory import Supermemory
import os

client = Supermemory(api_key=os.environ.get("SUPERMEMORY_API_KEY"))

connection = client.connections.create(
'google-drive',
redirect_url='https://yourapp.com/auth/google-drive/callback',
container_tags=['user-123', 'gdrive-sync'],
document_limit=3000,
metadata={
'source': 'google-drive',
'department': 'engineering'
```
```

```
}
)

# Redirect user to Google OAuth
print(f'Redirect to: {connection.auth_link}')
print(f'Expires in: {connection.expires_in}')
```



```bash theme={null}
curl -X POST "https://api.supermemory.ai/v3/connections/google-drive" \
-H "Authorization: Bearer $SUPERMEMORY_API_KEY" \
-H "Content-Type: application/json" \
-d '{
"redirectUrl": "htt
... (truncated)
```

connectors notion

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Notion Connector

> Sync Notion pages, databases, and blocks with real-time webhooks and workspace integration

Connect Notion workspaces to automatically sync pages, databases, and content blocks into your
Supermemory knowledge base. Supports real-time updates, rich formatting, and database properties.

## Quick Setup

### 1. Create Notion Connection



```typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY!
});

const connection = await client.connections.create('notion', {
redirectUrl: 'https://yourapp.com/auth/notion/callback',
containerTags: ['user-123', 'notion-workspace'],
documentLimit: 2000,
metadata: {
source: 'notion',
workspaceType: 'team',
department: 'product'
}
```
```

```
});

// Redirect user to Notion OAuth
window.location.href = connection.authLink;
```



```python theme={null}
from supermemory import Supermemory
import os

client = Supermemory(api_key=os.environ.get("SUPERMEMORY_API_KEY"))

connection = client.connections.create(
'notion',
redirect_url='https://yourapp.com/auth/notion/callback',
container_tags=['user-123', 'notion-workspace'],
document_limit=2000,
metadata={
'source': 'notion',
'workspaceType': 'team',
'department': 'product'
}
)

# Redirect user to Notion OAuth
print(f'Redirect to: {connection.auth_link}')
```



```bash theme={null}
curl -X POST "https://api.supermemory.ai/v3/connections/notion" \
-H "Authorization: Bearer $SUPERMEMORY_API_KEY" \
-H "Content-Type: application/json" \
-d '{
"redirectUrl": "https://yourap
... (truncated)
```

connectors onedrive

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# OneDrive Connector

> Sync Microsoft Office documents from OneDrive with scheduled synchronization and business account
support

Connect OneDrive to automatically sync Word documents, Excel spreadsheets, and PowerPoint presentations
into your Supermemory knowledge base. Supports both personal and business accounts with scheduled
```

```
synchronization.

## Quick Setup

### 1. Create OneDrive Connection


```typescript Typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY!
});

const connection = await client.connections.create('onedrive', {
redirectUrl: 'https://yourapp.com/auth/onedrive/callback',
containerTags: ['user-123', 'onedrive-sync'],
documentLimit: 1500,
metadata: {
source: 'onedrive',
accountType: 'business',
department: 'marketing'
}
});

// Redirect user to Microsoft OAuth
window.location.href = connection.authLink;
// Output: Redirects to OAuth provider
// Output: Redirects to https://login.microsoftonline.com/oauth2/authorize?...
```

```python Python theme={null}
from supermemory import Supermemory
import os

client = Supermemory(api_key=os.environ.get("SUPERMEMORY_API_KEY"))

connection = client.connections.create(
'onedrive',
redirect_url='https://yourapp.com/auth/onedrive/callback',
container_tags=['user-123', 'onedrive-sync'],
document_limit=1500,
metadata={
'source': 'onedrive',
'accountType': 'business',
'department': 'marketing'
}
)

# Redirect user to Microsoft OAuth
print(f'Redirect to: {connection.auth_link}')
# Output: Redirect to: https://oauth.provider.com/...
# Output: Redirect to: https://login.microsoftonline.com/oauth2/authorize?..
... (truncated)
```
```

connectors overview

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Connectors Overview

> Integrate Google Drive, Gmail, Notion, OneDrive, GitHub and Web Crawler to automatically sync documents
into your knowledge base

Connect external platforms to automatically sync documents into supermemory. Supported connectors include
Google Drive, Gmail, Notion, OneDrive, GitHub and Web Crawler with real-time synchronization and
intelligent content processing.

## Supported Connectors



**Google Docs, Slides, Sheets**

Real-time sync via webhooks. Supports shared drives, nested folders, and collaborative documents.



**Email Threads**

Real-time sync via Pub/Sub webhooks. Syncs threads with full conversation history and metadata.



**Pages, Databases, Blocks**

Instant sync of workspace content. Handles rich formatting, embeds, and database properties.



**Word, Excel, PowerPoint**

Scheduled sync every 4 hours. Supports personal and business accounts with file versioning.



**GitHub Repositories**

Real-time incremental sync via webhooks. Supports documentation files in repositories.



**Web Pages, Documentation**

Crawl websites automatically with robots.txt compliance. Scheduled recrawling keeps content up to date.



## Quick Start
```

```
### 1. Create Connection


```typescript Typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY!
});

const connection = await client.connections.create('notion', {
redirectUrl: 'https://yourapp.com/callback',
containerTags: ['user-123', 'workspace-alpha'],
documentLimit: 5000,
metadata: { department: 'sales' }
});

// Redirect user to complete OAuth
console.log('Auth URL:', connection.authLink);
console.log('Expires in:', connection.
... (truncated)
```

connectors s3

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# S3 Connector

> Connect Amazon S3 or S3-compatible storage to sync files into your Supermemory knowledge base

Connect Amazon S3 buckets or S3-compatible storage services (MinIO, DigitalOcean Spaces, Cloudflare R2)
to sync files into your Supermemory knowledge base.


The S3 connector requires a **Scale Plan** or higher. You can also create S3 connections directly from the
[Supermemory Console](https://console.supermemory.ai).


## Quick Setup



```typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY!
});

const connection = await client.connections.create('s3', {
accessKeyId: process.env.AWS_ACCESS_KEY_ID!,
```

```
secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY!,
bucket: 'my-documents-bucket',
region: 'us-east-1',
containerTags: ['org-123']
});
```



```python theme={null}
from supermemory import Supermemory
import os

client = Supermemory(api_key=os.environ.get("SUPERMEMORY_API_KEY"))

connection = client.connections.create(
's3',
access_key_id=os.environ.get("AWS_ACCESS_KEY_ID"),
secret_access_key=os.environ.get("AWS_SECRET_ACCESS_KEY"),
bucket='my-documents-bucket',
region='us-east-1',
container_tags=['org-123', 's3-sync']
)
```



```bash theme={null}
curl -X POST "https://api.supermemory.ai/v3/connections/s3" \
-H "Authorization: Bearer $SUPERMEMORY_API_KEY" \
-H "Content-Type: application/json" \
-d '{
"accessKeyId": "AKIAIOSFODNN7EXAMPLE",
"secretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
"bucket": "my-documents-bucket",
"region": "us-east-1",
"containerTags": ["org-123"
... (truncated)
```

connectors troubleshooting

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Connector Troubleshooting

> Diagnose and resolve common issues with Google Drive, Gmail, Notion, and OneDrive connectors

Quick guide to resolve common connector issues with authentication, syncing, and permissions.

## Quick Health Check
```

Check if your connectors are working properly:

```typescript TypeScript theme={null}
const connections = await client.connections.list({
containerTags: ['user-123']
});

connections.forEach(conn => {
console.log(`${conn.provider}: ${conn.email} - Connected ${conn.createdAt}`);
});

// Check for stuck documents
const documents = await client.connections.listDocuments('notion', {
containerTags: ['user-123']
});

const failed = documents.filter(doc => doc.status === 'failed');
if (failed.length > 0) {
console.log(`⚠️ ${failed.length} documents failed to sync`);
}
```

```python Python theme={null}
connections = client.connections.list(container_tags=['user-123'])

for conn in connections:
print(f"{conn.provider}: {conn.email} - Connected {conn.created_at}")

# Check for stuck documents
documents = client.connections.list_documents(
'notion',
container_tags=['user-123']
)

failed = [doc for doc in documents if doc.status == 'failed']
if failed:
print(f"⚠️ {len(failed)} documents failed to sync")
```

```bash cURL theme={null}
# List all connections
curl -X POST "https://api.supermemory.ai/v3/connections/list" \
-H "Authorization: Bearer $SUPERMEMORY_API_KEY" \
-H "Content-Type: application/json" \
-d '{"containerTags": ["user-123"]}'

# Check document status
curl -X POST "https://api.supermemory.ai/v3/documents/list" \
-H "Authorization: Bearer $SUPERMEMORY_API_KEY" \
-H "Content-Type: application/json" \
-d '{"containerTags": ["user-123
... (truncated)
```

connectors web-crawler

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Web Crawler Connector

> Crawl and sync websites automatically with scheduled recrawling and robots.txt compliance

Connect websites to automatically crawl and sync web pages into your Supermemory knowledge base. The web
crawler respects robots.txt rules, includes SSRF protection, and automatically recrawls sites on a
schedule.


The web crawler connector requires a **Scale Plan** or **Enterprise Plan**.


## Quick Setup

### 1. Create Web Crawler Connection



```typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY!
});

const connection = await client.connections.create('web-crawler', {
redirectUrl: 'https://yourapp.com/callback',
containerTags: ['user-123', 'website-sync'],
documentLimit: 5000,
metadata: {
startUrl: 'https://docs.example.com'
}
});

// Web crawler doesn't require OAuth - connection is ready immediately
console.log('Connection ID:', connection.id);
console.log('Connection created:', connection.createdAt);
// Note: connection.authLink is undefined for web-crawler
```



```python theme={null}
from supermemory import Supermemory
import os

client = Supermemory(api_key=os.environ.get("SUPERMEMORY_API_KEY"))

connection = client.connections.create(
'web-crawler',
redirect_url='https://yourapp.com/callback',
```
```

```
container_tags=['user-123', 'website-sync'],
document_limit=5000,
metadata={
'startUrl': 'https://docs.example.com'
}
)

# Web crawler doesn't require OAuth - connection is ready immediately
print(f'Connection ID: {connection.id}')
print(f'Connection created: {connection.created_at}')
# Note:
... (truncated)
```

memory-api connectors managing-resources

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Managing Connection Resources

> Get and configure resources for connections that support resource management


**Currently Available for GitHub:** Resource management endpoints are currently only available for the
GitHub connector. These endpoints allow you to select which repositories to sync before syncing begins.


Some connectors require you to select which resources (e.g., repositories) to sync before syncing begins.
Use these generic endpoints to list and configure resources for connections that support resource
management.

## Get Resources

`GET /v3/connections/:connectionId/resources`

Get available resources (e.g., repositories, folders) for a connection using stored credentials.


```typescript Typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory({
apiKey: process.env['SUPERMEMORY_API_KEY'],
});

// Get resources with pagination
const response = await fetch(
`https://api.supermemory.ai/v3/connections/${connectionId}/resources?page=1&per_page=30`,
{
headers: {
'Authorization': `Bearer ${process.env.SUPERMEMORY_API_KEY}`,
},
```
```

```
}
);

const data = await response.json();
console.log('Resources:', data.resources);
console.log('Total count:', data.total_count);
```

```python Python theme={null}
import requests

url = f"https://api.supermemory.ai/v3/connections/{connection_id}/resources"
params = {
"page": 1,
"per_page": 30
}
headers = {
"Authorization": f"Bearer {api_key}",
}

response = requests.get(url, params=params, headers=headers)
data = response.json()

print(f"Resources: {data['resources']}")
print(f"Total count: {data.get('total_count')}")
```

```bash cURL theme={null}
curl -X GET \
"https://api.supermemory.ai/v3/connectio
... (truncated)
```

Integrations

Pages	Content Size
21	182,908 characters

integrations agent-framework

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Microsoft Agent Framework

> Add persistent memory to Microsoft Agent Framework agents with Supermemory
```

Microsoft's [Agent Framework](https://github.com/microsoft/agent-framework) is a Python framework for building AI agents with tools, handoffs, and context providers. Supermemory integrates natively as a context provider, tool set, or middleware — so your agents remember users across sessions.

What you can do

* Automatically inject user memories before every agent run (context provider)
* Give agents tools to search and store memories on their own
* Intercept chat requests to add memory context via middleware
* Combine all three for maximum flexibility

Setup

Install the package:

```bash theme={null}
pip install --pre supermemory-agent-framework
```

Or with uv:

```bash theme={null}
uv add --prerelease=allow supermemory-agent-framework
```

The `--pre` / `--prerelease=allow` flag is required because `agent-framework-core` depends on pre-release versions of Azure packages.

Set up your environment:

```bash theme={null}
# .env
SUPERMEMORY_API_KEY=your-supermemory-api-key
OPENAI_API_KEY=your-openai-api-key
```

Get your Supermemory API key from console.supermemory.ai.

***

Connection

All integration points share a single `AgentSupermemory` connection. This ensures the same API client, container tag, and conversation ID are used across middleware, tools, and context providers.

```python theme={null}
from supermemory_agent_framework import AgentSupermemory

conn = AgentSupermemory(
api_key="your-supermemory-api-key", # or set SUPERMEMORY_API_KEY env var
container_tag="user-123", # memory scope (e.g., user ID)
conversation_id="session-abc", # optional, auto-generated if omitted
entity_context="The user is
... (truncated)
```

integrations agno

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Agno

> Add persistent memory to Agno agents with Supermemory

Agno agents are stateless by default. Each conversation starts fresh. Supermemory changes that - your
agents can remember users, recall past conversations, and build on previous interactions.

## What you can do

* Give agents access to user profiles and conversation history
* Store agent interactions for future sessions
* Let agents search memories to answer questions with context

## Setup

Install the packages:

```bash theme={null}
pip install agno supermemory python-dotenv
```

Set up your environment:

```bash theme={null}
# .env
SUPERMEMORY_API_KEY=your-supermemory-api-key
OPENAI_API_KEY=your-openai-api-key
```

Get your Supermemory API key from console.supermemory.ai.

Basic integration

Fetch user context before running an agent, then store the interaction after.

```python theme={null}
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from supermemory import Supermemory
from dotenv import load_dotenv

load_dotenv()

memory = Supermemory()

def get_user_context(user_id: str, query: str) -> str:
"""Pull user profile and relevant memories."""
result = memory.profile(container_tag=user_id, q=query)

static = result.profile.static or []
dynamic = result.profile.dynamic or []
```
```

```
memories = result.search_results.results if result.search_results else []

return f"""
User background:
{chr(10).join(static) if static else 'No profile yet.'}

Recent activity:
{chr(10).join(dynamic) if dynamic else 'Nothing recent.'}

Related memories:
{chr(10).join([m.memory or m.chunk for m in memories[:5]]) if memories else 'None.'}
"""

def create_agent(user_id: str, task: str) -> Agent:
"""Create an agent with user context."""
context = get_user_context
... (truncated)
```

integrations ai-sdk

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Vercel AI SDK

> Use Supermemory with Vercel AI SDK for seamless memory management

The Supermemory AI SDK provides native integration with Vercel's AI SDK through two approaches: **User
Profiles** for automatic personalization and **Memory Tools** for agent-based interactions.


Check out the NPM page for more details


## Installation

```bash theme={null}
npm install @supermemory/tools
```

Quick Comparison

Approach	Use Case	Setup
User Profiles	Personalized LLM responses with automatic user context	Simple middleware
Memory Tools	AI agents that need explicit memory control	Tool definitions

***

User Profiles with Middleware

Automatically inject user profiles into every LLM call for instant personalization.
```

```typescript theme={null}
import { generateText } from "ai"
import { withSupermemory } from "@supermemory/tools/ai-sdk"
import { openai } from "@ai-sdk/openai"

const modelWithMemory = withSupermemory(openai("gpt-5"), "user-123")

const result = await generateText({
model: modelWithMemory,
messages: [{ role: "user", content: "What do you know about me?" }]
})
```


Memory saving is disabled by default. The middleware only retrieves existing memories. To
automatically save new memories:

```typescript theme={null}
const modelWithMemory = withSupermemory(openai("gpt-5"), "user-123", {
addMemory: "always"
})
```


Memory Search Modes

Profile Mode (Default) - Retrieves the user's complete profile:

```typescript theme={null}
const model = withSupermemory(openai("gpt-4"), "user-123", { mode: "profile" })
```

Query Mode - Searches memories based on
... (truncated)

integrations claude-code

> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

Claude Code

> Claude Code Supermemory Plugin — persistent memory across coding sessions

This integration requires the **Supermemory Pro plan**. [Upgrade
here](https://console.supermemory.ai/billing).

[Claude-Supermemory](https://github.com/supermemoryai/claude-supermemory) is a Claude Code plugin that gives your AI persistent memory across sessions. Your agent remembers what you worked on — across sessions, across projects.

Get Your API Key

Create a Supermemory API key from the [API Keys](https://console.supermemory.ai/keys) page, then add it to your shell profile so it persists across sessions:

```bash theme={null}
echo 'export SUPERMEMORY_CC_API_KEY="sm_..."' >> ~/.zshrc
source ~/.zshrc
```

```bash theme={null}
echo 'export SUPERMEMORY_CC_API_KEY="sm_..."' >> ~/.bashrc
source ~/.bashrc
```

```powershell theme={null}
[System.Environment]::SetEnvironmentVariable("SUPERMEMORY_CC_API_KEY", "sm_...", "User")
```

Restart your terminal after running this.

Install the Plugin

```bash theme={null}
# Add the plugin marketplace
/plugin marketplace add supermemoryai/claude-supermemory

# Install the plugin
/plugin install claude-supermemory
```

How It Works

Once installed, the plugin runs automatically:

* **Context Injection** — On session start, relevant memories are fetched and injected into Claude's context. This includes user preferences, project knowledge, and past interactions.
* **Auto-Capture** — Tool usage is captured and stored during the session for future context.

What Gets Captured

Tool	Captured As

```
| Edit | `Edited src/auth.ts: "old code..." → "new code..."` |
| Write
... (truncated)
```

integrations claude-memory

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Claude Memory Tool

> Use Claude's native memory tool with Supermemory as the backend

Claude has a native memory tool that allows it to store and retrieve information across conversations.
Supermemory provides a backend implementation that maps Claude's memory commands to persistent storage.


This integration works with Claude's built-in `memory` tool type, introduced in the Anthropic API. It
requires the `context-management` beta flag.


## Installation

```bash theme={null}
npm install @supermemory/tools @anthropic-ai/sdk
```

Quick Start

```typescript theme={null}
import Anthropic from "@anthropic-ai/sdk"
import { createClaudeMemoryTool } from "@supermemory/tools/claude-memory"

const anthropic = new Anthropic()

const memoryTool = createClaudeMemoryTool(process.env.SUPERMEMORY_API_KEY!, {
projectId: "my-app",
})

async function chatWithMemory(userMessage: string) {
// Send message to Claude with memory tool
const response = await anthropic.beta.messages.create({
model: "claude-sonnet-4-5",
max_tokens: 2048,
messages: [{ role: "user", content: userMessage }],
tools: [{ type: "memory_20250818", name: "memory" }],
betas: ["context-management-2025-06-27"],
})

// Handle any memory tool calls
const toolResults = []
for (const block of response.content) {
```
```

```
if (block.type === "tool_use" && block.name === "memory") {
const toolResult = await memoryTool.handleCommandForToolResult(
block.input as any,
block.id
)
toolResults.push(toolResult)
}
}

// Send tool results back to Claude if needed
if (toolResults.length > 0) {
const finalResponse = await anthropic.beta.messages.create({
model: "claude-sonnet-4-5",
max_tokens: 2048,
messages: [
{ role: "user", content: userMessage }
... (truncated)
```

integrations crewai

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# CrewAI

> Add persistent memory to CrewAI agents with Supermemory

CrewAI agents don't remember anything between runs by default. Supermemory fixes that. You get a memory
layer that stores what happened, who the user is, and what they care about. Your crews can pick up where
they left off.

## What you can do

* Give agents access to user preferences and past interactions
* Store crew outputs so future runs can reference them
* Search memories to give agents relevant context before they start

## Setup

Install the required packages:

```bash theme={null}
pip install crewai supermemory python-dotenv
```

Configure your environment:

```bash theme={null}
# .env
SUPERMEMORY_API_KEY=your-supermemory-api-key
OPENAI_API_KEY=your-openai-api-key
```
```

```
Get your Supermemory API key from [console.supermemory.ai](https://console.supermemory.ai).

## Basic Integration

Initialize Supermemory and inject user context into your agent's backstory:

```python theme={null}
import os
from crewai import Agent, Task, Crew, Process
from supermemory import Supermemory
from dotenv import load_dotenv

load_dotenv()

memory = Supermemory()

def build_context(user_id: str, query: str) -> str:
"""Fetch user profile and relevant memories."""
result = memory.profile(container_tag=user_id, q=query)

static = result.profile.static or []
dynamic = result.profile.dynamic or []
memories = result.search_results.results if result.search_results else []

return f"""
User Profile:
{chr(10).join(static) if static else 'No profile data.'}

Current Context:
{chr(10).join(dynamic) if dynamic else 'No recent activity.'}

Relevant History:
{chr(10).join([m.memory or m.chunk for m in memories[:5]]) if memories else 'None.'}
"""

def create_agent_with_memory(user_id: str, role: str, goal: str, query: str) ->
... (truncated)
```

integrations hermes

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Hermes

> Hermes Supermemory Plugin — semantic memory, profiles, and search across Telegram, Discord, Slack, CLI,
and more

[Hermes agent](https://github.com/NousResearch/hermes-agent) ships a native **supermemory** memory
provider: semantic long-term memory, profile recall, search, explicit memory tools, and session-aware
ingest — not a bolt-on script the model might skip. Hermes runs across Telegram, Discord, Slack,
```
```

```
WhatsApp, Signal, and the CLI from one gateway.

Hermes also includes built-in **`MEMORY.md`** and **`USER.md`**. supermemory adds **structure and
isolation** (profile-scoped and optional multi-container tags) plus retrieval that goes beyond a single
flat file.

## Get Your API Key

Create a supermemory API key from the [app with plugins deep
link](https://app.supermemory.ai/?plugins=true) (or the [API Keys](https://console.supermemory.ai/keys)
page in the console). During `hermes memory setup` you can paste it when prompted, or persist it in your
environment:

## Install the memory provider

```bash theme={null}
pip install supermemory
```

```bash theme={null}
hermes memory setup
```

Select **supermemory** when prompted and paste your API key.

Or set the provider and key manually:

```bash theme={null}
hermes config set memory.provider supermemory
echo 'SUPERMEMORY_API_KEY=sm_...' >> ~/.hermes/.env
```

(Adjust the path if your Hermes home directory differs.)

How It Works

Once configured, the provider runs through Hermes's normal memory lifecycle:

* **Prefetch** — Relevant memory context can be loaded before each turn.
* **Turn capture** — Cleaned user/assistant turns can be stored after each completed response.
* **Session ingest** — The full session can be ingested at session end for richer graph updates.
* **Explicit tools** — Search, store, forget, and profi
... (truncated)
```

integrations langchain

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# LangChain

> Build AI agents with persistent memory using LangChain and Supermemory
```

Build AI applications with LangChain that remember context across conversations. Supermemory handles memory storage, retrieval, and user profiling while LangChain manages your conversation flow.

Overview

This guide shows how to integrate Supermemory with LangChain to create AI agents that:

* Maintain user context through automatic profiling
* Store and retrieve relevant memories semantically
* Personalize responses based on conversation history

Setup

Install the required packages:

```bash theme={null}
pip install langchain langchain-openai supermemory python-dotenv
```

Configure your environment:

```bash theme={null}
# .env
SUPERMEMORY_API_KEY=your-supermemory-api-key
OPENAI_API_KEY=your-openai-api-key
```

Get your Supermemory API key from console.supermemory.ai.

Basic Integration

Initialize both clients and set up a simple chat function with memory:

```python theme={null}
import os
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from supermemory import Supermemory
from dotenv import load_dotenv

load_dotenv()

# Initialize clients
llm = ChatOpenAI(model="gpt-4o")
memory = Supermemory()

def chat(user_id: str, message: str) -> str:
# 1. Get user profile for context
profile_result = memory.profile(container_tag=user_id, q=message)

# 2. Build context from profile
static_facts = profile_result.profile.static or []
dynamic_context = profile_result.profile.dynamic or []
search_results = profile_result.search_results.results if profile_result.search_results else []

context = f"""
```

```
User Background:
... (truncated)
```

integrations langgraph

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# LangGraph

> Add persistent memory to LangGraph agents with Supermemory

Build stateful agents with LangGraph that remember context across sessions. Supermemory handles memory
storage and retrieval while LangGraph manages your graph-based conversation flow.

## Overview

This guide shows how to integrate Supermemory with LangGraph to create agents that:

* Maintain user context through automatic profiling
* Store and retrieve relevant memories at each node
* Use conditional logic to decide what's worth remembering
* Combine short-term (session) and long-term (cross-session) memory

## Setup

Install the required packages:

```bash theme={null}
pip install langgraph langchain-openai supermemory python-dotenv
```

Configure your environment:

```bash theme={null}
# .env
SUPERMEMORY_API_KEY=your-supermemory-api-key
OPENAI_API_KEY=your-openai-api-key
```

Get your Supermemory API key from console.supermemory.ai.

Basic integration

A minimal agent that fetches user context before responding and stores the conversation after:

```python theme={null}
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage
```
```

```
from supermemory import Supermemory
from dotenv import load_dotenv

load_dotenv()

llm = ChatOpenAI(model="gpt-4o")
memory = Supermemory()

class State(TypedDict):
messages: Annotated[list, add_messages]
user_id: str

def agent(state: State):
user_id = state["user_id"]
messages = state["messages"]
user_query = messages[-1].content

# Fetch user profile with relevant memories
profile_result = memory.profile(container_tag=user_id, q=user_query)
... (truncated)
```

integrations mastra

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Mastra

> Add persistent memory to Mastra AI agents with Supermemory processors

Integrate Supermemory with [Mastra](https://mastra.ai) to give your AI agents persistent memory. Use the
`withSupermemory` wrapper for zero-config setup or processors for fine-grained control.


Check out the NPM page for more details


## Installation

```bash theme={null}
npm install @supermemory/tools @mastra/core
```

Quick Start

Wrap your agent config with `withSupermemory` to add memory capabilities:

```typescript theme={null}
import { Agent } from "@mastra/core/agent"
import { withSupermemory } from "@supermemory/tools/mastra"
import { openai } from "@ai-sdk/openai"
```
```

```
// Create agent with memory-enhanced config
const agent = new Agent(withSupermemory(
{
id: "my-assistant",
name: "My Assistant",
model: openai("gpt-4o"),
instructions: "You are a helpful assistant.",
},
"user-123", // containerTag - scopes memories to this user
{
mode: "full",
addMemory: "always",
threadId: "conv-456",
}
))

const response = await agent.generate("What do you know about me?")
```


Memory saving is disabled by default. The wrapper only retrieves existing memories. To automatically save conversations:

```typescript theme={null}
const agent = new Agent(withSupermemory(
{ id: "my-assistant", model: openai("gpt-4o"), ... },
"user-123",
{
addMemory: "always",
threadId: "conv-456" // Required for conversation grouping
}
))
```


***

How It Works

The Mastra integration uses Mastra's native [Processor](https://mastra.ai/docs/agents/processors) interface:

1. **Input Processor** - Fetches relevant memories from Supermemory and injects them into the system prompt before the LLM call
2. **Output Processor** - Optionally saves the conversa
... (truncated)
```

# integrations memory-graph

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.
```
```

Memory Graph

> Interactive visualization for documents, memories and connections

Memory Graph is a React component that visualizes your Supermemory documents and memories as an interactive network. Documents appear as rectangular nodes, memories as hexagonal nodes, and connections between them show relationships and similarity.

Check out the NPM page for more details

Installation

```bash theme={null}
npm install @supermemory/memory-graph
```

Requirements: React 18.0.0 or higher

Quick Start

```tsx theme={null}
'use client'; // For Next.js App Router

import { MemoryGraph } from '@supermemory/memory-graph';
import type { DocumentWithMemories } from '@supermemory/memory-graph';
import { useEffect, useState } from 'react';

export default function GraphPage() {
const [documents, setDocuments] = useState([]);
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
fetch('/api/graph')
.then(res => res.json())
.then(data => {
setDocuments(data.documents);
setIsLoading(false);
})
.catch(err => {
setError(err);
setIsLoading(false);
});
}, []);

return (



);
}
```

Backend API Route

```
Create an API route to fetch documents from Supermemory:


```typescript Next.js App Router theme={null}
// app/api/graph/route.ts
import { NextResponse } from 'next/server';

export async function GET() {
const response = await fetch('https://api.supermemory.ai/v3/documents/documents', {
method: 'POST',
headers: {
'Content-Type': 'application/json',
'Authorization': `Bearer ${process.env.SUPERMEMORY_API_KEY}`,
},
body: JSON.stringify({
page: 1,
limit: 500,
sort: 'crea
... (truncated)
```

integrations n8n

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# n8n

> Automate knowledge management with Supermemory in n8n workflows

Connect Supermemory to your n8n workflows to build intelligent automation workflows and agents that
leverage your full knowledge base.

## Quick Start

### Prerequisites

* n8n instance (self-hosted or cloud)
* Supermemory API key ([get one here](https://console.supermemory.com/settings))
* Basic understanding of n8n workflows

### Setting Up the HTTP Request Node

The Supermemory integration in n8n uses the HTTP Request node to interact with the Supermemory API. Here's
how to configure it:

1. Add an **HTTP Request** node to your workflow (Core > HTTP Request)

2. Set the **Method** to `POST`
3. Set the **URL** to the appropriate Supermemory API endpoint:
* Add memory: `https://api.supermemory.ai/v3/documents`
* Search memories: `https://api.supermemory.ai/v4/search`
4. For authentication, select **Generic Credential Type** and then **Bearer Auth**
```
```

```
5. Click on **Create New Credential** and paste the Supermemory API Key in the Bearer Token field.

6. Check **Send Body** and select **JSON** as the Body Content Type. The fields depend on what API
endpoint you're sending the request to. You can find detailed step-by-step examples below.

## Step-by-Step Tutorial

In this tutorial, we'll create a workflow that automatically adds every email from Gmail to your
Supermemory knowledge base. We'll use the HTTP Request node to send email data to Supermemory's API,
creating a searchable archive of all your communications.

### Adding Gmail Emails to Supermemory

Follow these steps to build a workflow that captures and stores your Gmail messages:

#### Step 1: Set Up Gmail Trigger



1. **Add a Gmail Trigger node** to your workflow
2. Configure your Gmail credentials (OAuth2 recommended)
3. Set the trigger to **Mes
... (truncated)
```

integrations openai-agents-sdk

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# OpenAI Agents SDK

> Add persistent memory to OpenAI agents with Supermemory

OpenAI's Agents SDK gives you a straightforward way to build agents with tools, handoffs, and guardrails.
But agents don't remember users between sessions. Supermemory adds that missing piece: your agents can
store what they learn and recall it later.

## What you can do

* Pull user profiles and relevant memories before an agent runs
* Store agent outputs and decisions for future sessions
* Give agents tools to search and add memories on their own

## Setup

Install the packages:

```bash theme={null}
pip install openai-agents supermemory python-dotenv
```


Set up your environment:
```

```bash theme={null}
# .env
SUPERMEMORY_API_KEY=your-supermemory-api-key
OPENAI_API_KEY=your-openai-api-key
```

Get your Supermemory API key from console.supermemory.ai.

Basic integration

The simplest approach: fetch user context and pass it in the agent's instructions.

```python theme={null}
import os
from agents import Agent, Runner
from supermemory import Supermemory
from dotenv import load_dotenv

load_dotenv()

memory = Supermemory()

def get_user_context(user_id: str, query: str) -> str:
"""Fetch profile and relevant memories for a user."""
result = memory.profile(container_tag=user_id, q=query)

static = result.profile.static or []
dynamic = result.profile.dynamic or []
memories = result.search_results.results if result.search_results else []

return f"""
User background:
{chr(10).join(static) if static else 'No profile yet.'}

Current focus:
{chr(10).join(dynamic) if dynamic else 'No recent activity.'}

Related memories:
{chr(10).join([m.memory or m.chunk for m in memories[:5]]) if memories else 'None.'}
"""

def create_agent(user_id: str, task: str) -> Agen
... (truncated)
```

integrations openai

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# OpenAI SDK
```

```
> Memory tools for OpenAI function calling with Supermemory integration

Add memory capabilities to the official OpenAI SDKs using Supermemory. Two approaches available:

1. **`withSupermemory` wrapper** - Automatic memory injection into system prompts (zero-config)
2. **Function calling tools** - Explicit tool calls for search/add memory operations


**New to Supermemory?** Start with `withSupermemory` for the simplest integration. It automatically
injects relevant memories into your prompts.




Check out the NPM page for more details



Check out the PyPI page for more details



***

## withSupermemory Wrapper

The simplest way to add memory to your OpenAI client. Wraps your client to automatically inject relevant
memories into system prompts.

### Installation

```bash theme={null}
npm install @supermemory/tools openai
```

Quick Start

```typescript theme={null}
import OpenAI from "openai"
import { withSupermemory } from "@supermemory/tools/openai"

const openai = new OpenAI()

// Wrap client with memory - memories auto-injected into system prompts
const client = withSupermemory(openai, "user-123", {
mode: "full", // "profile" | "query" | "full"
addMemory: "always", // "always" | "never"
})

// Use normally - memories are automatically included
const response = await client.chat.completions.create({
model: "gpt-5",
messages: [
{ role: "system", content: "You are a helpful assistant." },
{ role: "user", content: "What's my favorite programming language?" }
]
})
```
```

```
```

Configuration Options

```typescript theme={null}
const client = withSupermemory(openai, "user-123", {
// Memory search mode
mode: "full", // "profile" (user profile only), "query" (search only), "full" (both)

// Auto-save conversations as memories

... (truncated)
```

integrations openclaw

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# OpenClaw

> OpenClaw Supermemory Plugin — works across Telegram, WhatsApp, Discord, Slack, and more


This integration requires the **Supermemory Pro plan**. [Upgrade
here](https://console.supermemory.ai/billing).


[OpenClaw](https://github.com/supermemoryai/openclaw-supermemory) is a multi-platform AI messaging
gateway that connects to WhatsApp, Telegram, Discord, Slack, iMessage, and other messaging channels. The
Supermemory plugin gives OpenClaw memory across every channel.

## Install the Plugin

Get started by installing the plugin with a single command.

```bash theme={null}
openclaw plugins install @supermemory/openclaw-supermemory
```

Restart OpenClaw after installing.

Setup

Run the setup command and enter your API key when prompted.

```bash theme={null}
openclaw supermemory setup
```

Enter your API key from console.supermemory.ai. That's it.
```

```
Configure all options interactively with the advanced setup command:

```bash theme={null}
openclaw supermemory setup-advanced
```

This lets you configure: container tag, auto-recall, auto-capture, capture mode, custom container tags,
and more.

See [Configuration Options](#configuration-options) for all available settings.

How It Works

Once installed, the plugin runs automatically with zero interaction.

* **Auto-Recall** — Before every AI turn, Supermemory is queried for relevant memories and the user's
profile. These are injected as context so the AI sees preferences, facts, and semantically similar past
conversations.
* **Auto-Capture** — After every AI turn, the conversation exchange is sent to Supermemory for extraction
and long-term storage. Supermemory handles deduplication and profile building.
* **Custom Container Tags** — When enabled via advance
... (truncated)
```

integrations opencode

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# OpenCode

> OpenCode Supermemory Plugin — persistent memory across coding sessions


This integration requires the **Supermemory Pro plan**. [Upgrade
here](https://console.supermemory.ai/billing).


[OpenCode-Supermemory](https://github.com/supermemoryai/opencode-supermemory) is an OpenCode plugin that
gives your AI persistent memory across sessions. Your agent remembers what you worked on — across
sessions, across projects.

## Get Your API Key

Create a Supermemory API key from the [API Keys](https://console.supermemory.ai/keys) page, then add it
to your shell profile so it persists across sessions:



```bash theme={null}
```
```

```
echo 'export SUPERMEMORY_API_KEY="sm_..."' >> ~/.zshrc
source ~/.zshrc
```



```bash theme={null}
echo 'export SUPERMEMORY_API_KEY="sm_..."' >> ~/.bashrc
source ~/.bashrc
```



```powershell theme={null}
[System.Environment]::SetEnvironmentVariable("SUPERMEMORY_API_KEY", "sm_...", "User")
```

Restart your terminal after running this.

Install the Plugin

```bash theme={null}
bunx opencode-supermemory@latest install
```

For LLM agents (non-interactive):

```bash theme={null}
bunx opencode-supermemory@latest install --no-tui
```

Ensure your `~/.config/opencode/opencode.jsonc` contains:

```json theme={null}
{
"plugin": ["opencode-supermemory"]
}
```

How It Works

Once installed, the plugin runs automatically:

* **Context Injection** — On session start, relevant memories are fetched and injected into the agent's context. This includes user preferences, project knowledge, and past interactions.
* **Keyword Detection** — Phrases like "remember" or "save this" trigger automatic storage.
* **Smart Compaction** — At 80% context capacity, sessions are summarized and saved as memories.
* **Privacy
... (truncated)
```

# integrations pipecat
```

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Pipecat

> Integrate Supermemory with Pipecat for conversational memory in voice AI agents

Supermemory integrates with [Pipecat](https://github.com/pipecat-ai/pipecat), providing long-term memory
capabilities for voice AI agents. Your Pipecat applications will remember past conversations and provide
personalized responses based on user history.

## Installation

To use Supermemory with Pipecat, install the required dependencies:

```bash theme={null}
pip install supermemory-pipecat
```

Set up your API key as an environment variable:

```bash theme={null}
export SUPERMEMORY_API_KEY=your_supermemory_api_key
```

You can obtain an API key from console.supermemory.ai.

Configuration

Supermemory integration is provided through the `SupermemoryPipecatService` class in Pipecat:

```python theme={null}
from supermemory_pipecat import SupermemoryPipecatService
from supermemory_pipecat.service import InputParams

memory = SupermemoryPipecatService(
api_key=os.getenv("SUPERMEMORY_API_KEY"),
user_id="unique_user_id",
session_id="session_123",
params=InputParams(
mode="full", # "profile" | "query" | "full"
search_limit=10, # Max memories to retrieve
search_threshold=0.1, # Relevance threshold (0.0-1.0)
system_prompt="Based on previous conversations:\n\n",
),
)
```

Pipeline Integration

The `SupermemoryPipecatService` should be positioned between your context aggregator and LLM service in
the Pipecat pipeline:

```python theme={null}
pipeline = Pipeline([
transport.input(),
```
```

```
stt, # Speech-to-text
context_aggregator.user(),
memory, #
Full working example with Gemini Live, including frontend and backend code.
```

integrations supermemory-sdk

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Supermemory SDK

> Official Python and JavaScript SDKs for Supermemory




pip install supermemory




npm install supermemory






## Installation

```bash theme={null}
npm install supermemory
```

Quick Start

```typescript theme={null}
import Supermemory from 'supermemory';

const client = new Supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY, // Default, can be omitted
});

// Add a memory
await client.add({ content: "Meeting notes from Q1 planning", containerTags: ["user_123"] });

// Search memories
const response = await client.search.documents({
q: "planning notes",
containerTags: ["user_123"]
});
console.log(response.results);

// Get user profile
```
```

```
const profile = await client.profile({ containerTag: "user_123" });
console.log(profile.profile.static);
console.log(profile.profile.dynamic);
```

Common Operations

```typescript theme={null}
// Add with metadata
await client.add({
content: "Technical design doc",
containerTags: ["user_123"],
metadata: { category: "engineering", priority: "high" }
});

// Search with filters
const results = await client.search.documents({
q: "design document",
containerTags: ["user_123"],
filters: {
AND: [
{ key: "category", value: "engineering" }
]
}
});

// List documents
const docs = await client.documents.list({ containerTags: ["user_123"], limit: 10 });

// Delete a document
await client.documents.delete({ docId: "doc_123" });
```


Installation

```bash theme={null}
pip install supermemory
```

Quick Start

```python theme={null}
import os
from supermemory imp
... (truncated)
```

integrations viasocket

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.
```

```
# viaSocket

> Connect Supermemory with viaSocket to build automation flows using triggers, API tokens, and actions
like Gmail.

Connect Supermemory to viaSocket to build powerful automation flows — search your memory, trigger
actions, and wire up services like Gmail, all without writing code.

## Prerequisites

* A Supermemory API key ([get one here](https://console.supermemory.ai/settings))
* A viaSocket account

## Step-by-Step Tutorial



* Log in to your [Supermemory account](https://console.supermemory.ai).
* Go to **Settings → API Key**.
* Copy your Personal API key.
* Keep the key secure — treat it like a password.




* Click **Create New Flow** in your viaSocket dashboard.
* In the **Trigger** section, search for and select **Supermemory**.
* Choose a trigger — **Search Memory** or **Search User Profile**.




* Click **Connect to Supermemory**.
* Paste your Supermemory API key.
* Click **Save** to store the connection.
* Confirm the connection is successfully added before proceeding.




* Provide a **Query** — either a static value or a dynamic input from a previous step.
* Click **TEST** to run a sample and verify the output.
* Save the trigger once the test returns expected data.




* Click **Add Step**.
* Select **Gmail → Send Email**.
* Choose an existing Gmail connection or create a new one.
* Map the required fields:
* **To** — recipient email address
* **Subject** — email subject line
* **Message Body** — use the trigger's `body` object as dynamic input
* Click **Test** to send a test email.
* Confirm a **200** response status before saving.
```

```
* Click **GO LIVE** to activate your flow.
* Con
... (truncated)
```

integrations voltagent

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# VoltAgent

> Integrate Supermemory with VoltAgent for long-term memory in AI agents

Supermemory integrates with [VoltAgent](https://github.com/VoltAgent/voltagent), providing long-term
memory capabilities for AI agents. Your VoltAgent applications will remember past conversations and
provide personalized responses based on user history.


Check out the NPM page for more details


## Installation

```bash theme={null}
npm install @supermemory/tools @voltagent/core
```

Set up your API key as an environment variable:

```bash theme={null}
export SUPERMEMORY_API_KEY=your_supermemory_api_key
```

You can obtain an API key from console.supermemory.ai.

Quick Start

Supermemory provides a `withSupermemory` wrapper that enhances any VoltAgent agent config with automatic
memory retrieval and storage:

```typescript theme={null}
import { withSupermemory } from "@supermemory/tools/voltagent"
import { Agent } from "@voltagent/core"
import { openai } from "@ai-sdk/openai"

// Create an agent with Supermemory memory capabilities
const configWithMemory = withSupermemory({
agentConfig: {
```
```

```
name: "my-agent",
instructions: "You are a helpful assistant.",
model: openai("gpt-4o"),
},
containerTag: "user-123",
customId: "conversation-123",
})

const agent = new Agent(configWithMemory)

// Memories are automatically injected and saved
const result = await agent.generateText({
messages: [{ role: "user", content: "What's my name?" }],
})
```


Memory saving is enabled by default in the VoltAgent integration. To disable it:

```typescript theme={null}
const configWithMemory = withSupermemory({
agentConfig: {
name: "my-agent",
instructions: "You are a helpful assistant.",
model: openai("gpt-4o"),
},
containerTag
... (truncated)
```

integrations zapier

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Zapier

> Automate memory management with Supermemory in Zapier workflows

With Supermemory you can now easily add memory to your Zapier workflow steps. Here's how:

## Prerequisites

* A Supermemory API Key. Get yours [here](https://console.supermemory.ai)

## Step-by-step tutorial

For this tutorial, we're building a simple flow that adds incoming emails in Gmail to Supermemory.



Open your Zapier account and click on 'Zap' to make a new automation.
```

Add a new Gmail node that gets triggered on every new email. Connect to your Google account.

Now, add a new 'Code by Zapier' block. Set it up to run Python.

In the **Input Data** section, map the content field to the Gmail raw snippet.

Since we're ingesting data here, we'll use the add documents endpoint.

Add the following code block:

```python theme={null}
import requests

url = "https://api.supermemory.ai/v3/documents"

payload = { "content": inputData['content'], "containerTag": "gmail" }
headers = {
"Authorization": "Bearer YOUR_SM_API_KEY",
"Content-Type": "application/json"
}

response = requests.post(url, json=payload, headers=headers)

print(response.json())
```

The `inputData['content']` field maps to the Gmail content fetched from Zapier.

Sometimes Zapier might show an error on the first test run. It usually works right after. Weird bug, we know.

You can perform other operations like search, filtering, user profiles, etc., by using other Supermemory API endpoints which can be found in our API Reference tab.

Cookbook & Examples

Pages	Content Size
7	256,107 characters

cookbook ai-sdk-integration

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# AI SDK Integration

> Complete examples showing how to use Supermemory with Vercel AI SDK for building intelligent
applications

This page provides comprehensive examples of using Supermemory with the Vercel AI SDK, covering Memory
Tools and User Profiles approaches.

## Personal Assistant with Memory Tools

Build an AI assistant that remembers user preferences and past interactions:


```typescript Next.js API Route theme={null}
import { streamText } from 'ai'
import { createAnthropic } from '@ai-sdk/anthropic'
import { supermemoryTools } from '@supermemory/tools/ai-sdk'

const anthropic = createAnthropic({
apiKey: process.env.ANTHROPIC_API_KEY!
})

export async function POST(request: Request) {
const { messages } = await request.json()

const result = await streamText({
model: anthropic('claude-3-sonnet-20240229'),
messages,
tools: supermemoryTools(process.env.SUPERMEMORY_API_KEY!),
system: `You are a helpful personal assistant. When users share information about themselves,
remember it using the addMemory tool. When they ask questions, search your memories to provide
personalized responses. Always be proactive about remembering important details.`
})

return result.toAIStreamResponse()
}
```


```typescript Client Component theme={null}
'use client'
```
```

```
import { useChat } from 'ai/react'

export default function PersonalAssistant() {
const { messages, input, handleInputChange, handleSubmit } = useChat()

return (


{messages.map((message) => (

{message.content}

))}





)
}
```


Example conversation:

* User: "I'm allergic to peanuts and I love Italian food"
* AI: *Uses ad
... (truncated)
```

# Building an AI Compliance Chatbot With Supermemory and Google Drive

Building an AI Compliance Chatbot With Supermemory and Google Drive

Research

Pricing

Docs

Blog

Consumer

Plugins

Login

Developer Console

API keys, usage, docs
```

Personal App

Your memory dashboard

Research

Pricing

Docs

Blog

Consumer

Plugins

Supermemory App

Developer Console

Building an AI Compliance Chatbot With Supermemory and Google Drive

Naman Bansal

20 Jul 2025

• 15 min read

Contract compliance reviews are a serious drain on time and focus. It's a repetitive process that takes away from actual legal thinking, and the workflow is absolutely broken.

Files live in different places. You're never sure if you're reading the latest version. And no one has time to manually track every obligation, clause, or renewal date buried inside a hundred PDFs.

But what if you had an AI assistant to automatically handle it? One that connects directly to your (or your client's) Google Drive, stays in sync automatically, and lets you chat with your documents.

In this guide, we'll walk through building that AI assistant. It connects directly to your Google Drive, keeps itself updated as documents change, and lets you ask questions or run checks over the entire set of contracts without manual uploads or setup.

You'll useSupermemory to handle document syncing and memory, with a TypeScript backend and a Vite React frontend. You can also access all the code here:GitHub repo

Planning the app

Before jumping into the code, it helps to get clear on what we're actually building and why. Here's how I thought through the design:

1. What's the Assistant Actually Doing?

At a high level, it needs to:

- Connect to a Google Drive and access all relevant legal docs
- Automatically sync new or updated files

- Let the user ask questions about any of the documents
- Respond using context pulled from those files, plus memory of past chats

So the three core jobs are: connect, sync, and chat.

2. Why Supermemory?

Most “chat with your documents” setups require manually uploading files or rebuilding context every time. I wanted something that just stayed in sync without needing to re-upload or reprocess every time someone edits a contract.

Supermemory handles all that for me. It:

- Connects directly to Google Drive
- Watches for file changes and syncs them
- Adds memory and document context automatically to each AI call

That’s huge because I don’t have to manually store, chunk, or retrieve anything.

3. Tools Breakdown

Here’s how I picked the stack:

- Memory + File Syncing: Supermemory handles both, so no need to build my own database or embedding system.
- Chat + AI Responses: OpenAI (GPT-4o) via Supermemory, so memory is already baked into the API call.
- Backend: TypeScript with Express as it’s simple and flexible for API routes.
- Frontend: Vite + React + Tailwind because it’s fast to build and easy to style.

4. Assistant Workflow

Here’s the basic flow this app follows:

- Connect – User clicks a button → initiates Google Drive OAuth via Supermemory
- Sync – After uploading contracts, user clicks sync → Supermemory indexes the new files
- Chat – User sends a message → app hits the /chat API → Supermemory auto-adds document context → GPT-4o generates a useful, formatted reply

That’s the thinking. Now let’s dive right into the code.

Building the app: step-by-step

Project Setup

The first step in the tutorial is to set up our project. First, create a general directory that will house the project's backend and frontend files.

```
mkdir compliance-checker-assistant
```

Navigate to the compliance-checker-assistant directory in your terminal to continue with the tutorial.

```
cd compliance-checker-assistant
```

Create Assistant Backend

To set up our AI assistant backend. First, add a backend folder to your compliance-checker-assistant folder and navigate to the folder.

```
mkdir backend
cd backend
```

Initialize Node in the backend folder.

```
npm init -y
```

For the backend, we will be installing the following packages.

- express: To create our backend routes
- dotenv: To pass env variables
- openai: The OpenAI Typescript SDK
- supermemory: Supermemory AI Typescript SDK

Install the required packages with the following command -

```
npm install express supermemory openai
```

Install TypeScript and add types for Express.js and Node.

```
npm install --save-dev typescript @types/node @types/express
```

Initialize TypeScript:

```
npx tsc --init
```

This command will add a tsconfig.json file to your backend folder. Open the tsconfig.json file and add the following code to it:

```
"outDir": "./dist/",
```

Add the following scripts to your package.json file.

```
"scripts": {
"build": "tsc",
"start": "node dist/app.js",
"dev": "ts-node src/app.ts"
},
```

Create a .env file and add the following keys and their values to the file.

```
SUPERMEMORY_API_KEY=
OPENAI_API_KEY=
```

For instructions on how to retrieve your Supermemory AI api key, see - https://supermemory.ai/docs/model-enhancement/getting-started#get-your-supermemory-api-key

For instructions on how to retrieve your OpenAI api key, see - https://platform.openai.com/api-keys

Create your Assistant server

After setting up our backend, the next step is to create the assistant server. For this tutorial, we will be adding three routes to our backend.

- api/connect - This route will be used to connect to our Google Drive application with Supermemory AI. This will enable Supermemory AI to access the contract and compliance files stored in the drive.
- api/sync - This route will be used to sync Supermemory AI and our Google Drive. This is useful when we add new files to our drive and we need to keep our Supermemory files up to date.
- api/chat - This route will be used to chat with our AI assistant.

Add a src folder to your backend folder, and add an app.ts file to the folder.

```
mkdir src
touch app.ts
```

Add the following code sample to the app.ts file

```
import express from "express";
import "dotenv/config";
import chatRoutes from "./routes/chatRoutes";

const app = express();
const PORT = 3100;

app.use(express.json());

app.use("/api", chatRoutes);

app.listen(PORT, () =&gt; {
console.log(`Server running on http://localhost:${PORT}`);
});
```

In this code, we created our backend server using Express.js. We imported our .env file parameters using dotenv, imported our routes, and configured our server to listen on port 3100.

To create the required routes, add a routes folder to your src folder and add a chatRoutes.ts file to the folder.

```
mkdir routes
touch routes/chatRoutes.ts
```

Add the following code to the chatRoutes.ts file

```
import express, {
Request,
Response,
NextFunction,
RequestHandler,
} from "express";
import Supermemory from "supermemory";
import OpenAI from "openai";
const router = express.Router();
const client = new Supermemory({
```

```
apiKey: process.env["SUPERMEMORY_API_KEY"], // This is the default and can be omitted
});

const openai = new OpenAI({
apiKey: process.env.OPENAI_API_KEY,
baseURL: "https://api.supermemory.ai/v3/https://api.openai.com/v1",
defaultHeaders: {
"x-api-key": process.env.SUPERMEMORY_API_KEY,
},
});
```

In this code, we created an Express router, initialized a Supermemory client, and an OpenAI client. Note the OpenAI baseUrl and default headers; with this configuration, Supermemory automatically appends relevant memory and context to our conversation when interacting with OpenAI APIs.

Add the following code to the chatRoutes.ts file to create the api/connect route -

```
router.get("/connect", async (req: Request, res: Response) =&gt; {
try {
const connection = await client.connections.create("google-drive", {
redirectUrl: "http://localhost:5173",
});
res.status(201).json({
message: "authenticate",
url: connection.authLink,
});
} catch (error) {
console.error("Connection creation failed:", error);
res.status(500).json({
message: "Failed to initiate connection",
error: (error as Error).message,
});
}
});
```

First, we created a new Google Drive connection using the Suprememory SDK. An authentication link is then collected from the response and sent back as a response for the API.

Note that we added a redirectUrl parameter to the request. The authentication link will redirect to the redirect URL after the authentication process is completed.

Add the following code to the chatRoutes.ts file to create the api/sync route -

```
router.get("/sync", async (req: Request, res: Response) =&gt; {
try {
await client.connections.import("google-drive");
res.status(201).json({
message: "synced",
});
} catch (error) {
console.error("Sync failed:", error);
res.status(500).json({
message: "Failed to initiate sync",
error: (error as Error).message,
});
}
```

```
});
```

This code performs a connection sync between Supermemory and Google Drive. After completing the sync, the api returns a `synced` response.

Add the following code to the chatRoutes.ts file to create the api/chat route -

```
interface ChatRequestBody {
message: string;
}
interface ChatResponse {
response: string | null;
}
interface ErrorResponse {
error: string;
}

const chatHandler: RequestHandler&lt;
{},
ChatResponse | ErrorResponse,
ChatRequestBody
&gt; = async (req, res, next) =&gt; {
const { message } = req.body;
if (!message) {
res
.status(400)
.json({ error: 'Request body must include a "message" field.' });
return;
}

try {
const completion = await openai.chat.completions.create({
model: "gpt-4o",
user: "sheriff",
messages: [
{
role: "system",
content:
"You are ContractGuard, an AI assistant for contract and compliance reviews. When given any contract text,
you will:\n\n1. Highlight key obligations and risks.\n2. Point out missing or non-standard clauses.\n3.
Recommend fixes or best-practice language.\n4. Always format your answer in Markdown with headings and
bullet points.",
},
{ role: "user", content: message },
],
});

res.status(200).json({
response: completion.choices[0].message.content,
});
return;
} catch (err) {
console.error("Chat failed:", err);
res.status(500).json({ error: "Failed to initiate AI chat." });
return;
}
};
```

```
router.post("/chat", chatHandler);
```

The `chatHandler` function first extracts the user's message from the request body. It then passes the system our AI assistant system's prompt and the extracted user message to the OpenAI SDK to get a response.

Note that we are not adding our previous conversation manually to the messages; this is handled automatically by Supermemory. In addition to that, Supermemory also handles the AI's interaction with your Google Drive files.

Add the following to the end of the chatRoutes.ts file to export your routes.

```
export default router;
```

```
npm run build
npm run start
```

Create your app's frontend

After completing the assistant's backend, the next step is to create a chat interface frontend for the assistant. We will be using Vite React, TypeScript, and Tailwind CSS for our frontend code.

Create a Vite React TypeScript project named frontend.

```
npm create vite@latest frontend -- --template react-ts
```

```
cd frontend
npm install
```

```
npm install tailwindcss @tailwindcss/vite
```

```
import tailwindcss from '@tailwindcss/vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
plugins: [react(), tailwindcss()],
})
```

```
@import "tailwindcss";
```

```
npm run dev
```

The next step is to implement the chat interface, open your App.tsx file, and replace the code with the following code.

Let's start with the necessary imports:

```
import type React from 'react'

import { useState, useRef, useEffect } from 'react'
import ReactMarkdown from 'react-markdown'
import { Send, Bot, User, Loader2 } from 'lucide-react'
```

```
interface Message {
id: string
role: 'user' | 'assistant'
content: string
timestamp: Date
}

interface ConnectResponse {
message: string;
url?: string;
error?: string;
}

interface SyncResponse {
message: string;
error?: string;
}
```

The `Message` interface represents a single message in the conversation. It includes an `id` (used as a unique key when rendering messages), a `role` (either `user` or `assistant` so we can style them differently), the `content` (which is the actual message text), and a `timestamp`.

Every time a user sends a message or the assistant replies, we create a new `Message` object and add it to our state.

The `ConnectResponse` and `SyncResponse` types are used to type-check the data we get back from the backend API when connecting to Google Drive or syncing files to avoid runtime errors.

Now, let's set up the states and refs:

```
function App () {
const [messages, setMessages] = useState&lt;Message[]&gt;([])
const [input, setInput] = useState('')
const [isLoading, setIsLoading] = useState(false)
const messagesEndRef = useRef&lt;HTMLDivElement&gt;(null)
```

`input` stores whatever the user is currently typing. As the user types in the input box, this gets updated in real-time.

`isLoading` is a simple flag to tell us when the assistant is "thinking."

It's used to disable the input field and show a spinner in the chat UI so users don't accidentally submit multiple messages or get confused by lag.

`messagesRef` is a reference to an invisible div at the bottom of the chat window. We use this to automatically scroll the chat down to the latest message every time a new one is added.

Next, we'll write functions to connect to Google Drive and Sync the content, sending requests to our backend APIs:

```
const handleConnect = async () =&gt; {
try {
const res = await fetch('http://localhost:3100/api/connect', {
method: 'GET',
credentials: 'include',
```

```
});
const data: ConnectResponse = await res.json();

if (!res.ok) {
console.log(data.error ?? data.message);
} else if (data.url) {
// redirect to the OAuth URL
window.location.href = data.url;
}
} catch (err) {
console.log((err as Error).message);
}
};

const handleSync = async () =&gt; {
try {
const res = await fetch('http://localhost:3100/api/sync', {
method: 'GET',
credentials: 'include',
});
const data: SyncResponse = await res.json();

if (!res.ok) {
console.log(data.error ?? data.message);
} else {
console.log(data.message);
}
} catch (err) {
console.log((err as Error).message);
}
};
```

The `handleConnect` function makes a GET request to our backend route at `/api/connect`, it gets the `authLink` returned from the backend, and redirects the user to it. This is how the user grants permissions to Supermemory to read our GDrive files.

The `handleSync` function explicitly tells Supermemory to sync to Google Drive. Supermemory automatically does it whenever a new doc is uploaded, but in case you need manual refreshing, this function will work.

It also sends a GET request to our backend /api/sync endpoint and when it's successfully done, we log a simple confirmation message `{message: 'synced'}` in the console.

Now, let's handle the main logic of sending messages and receiving responses:

```
const scrollToBottom = () =&gt; {
messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' })
}

useEffect(() =&gt; {
scrollToBottom()
}, [messages])
```

The `scrollToBottom` function is a little helper that scrolls automatically down when a new message loads.

Now, the `chatHandler` is the main function that encompasses the logic of our app:

```
const chatHandler: RequestHandler&lt;{}, ChatResponse | ErrorResponse, ChatRequestBody&gt; = async (req,
res) =&gt; {
const { message } = req.body;
if (!message) {
res.status(400).json({ error: 'Request body must include a "message" field.' });
return;
}

try {
// Initial response: model decides if it should use the search tool
const initial = await openai.responses.create({
model: "gpt-4o",
instructions: `You are ContractGuard, an assistant for contract and compliance reviews.
Decide if you need to search documents to answer the user's question.
If yes, call the 'supermemory_search' tool with the query parameter.
Otherwise, provide an answer directly.`,
input: message,
tools: [
{
type: "function",
name: "supermemory_search",
description: "Search documents in Google Drive",
strict: true,
parameters: {
type: "object",
properties: {
query: {
type: "string",
description: "The search query for finding relevant documents"
},
},
required: ["query"],
additionalProperties: false,
},
},
],
tool_choice: "auto",
});

// Check if there's a function call in the output
const functionCall = initial.output.find(item =&gt; item.type === "function_call");

if (functionCall &amp;amp;&amp; functionCall.name === "supermemory_search") {
const args = JSON.parse(functionCall.arguments || "{}");

// Perform the search using supermemory client
let docs;
try {
// Use the correct Supermemory API method
docs = await client.search.execute({
q: args.query,
limit: 5
});
```

```
console.log(`▨ Search executed: "${args.query}"`);
} catch (searchError) {
console.error("Supermemory search failed:", searchError);
res.status(500).json({ error: "Failed to search docume
... (truncated)
```

First, as soon as a request hits '/chat', we pul message out of the JSON body. If it's missing, we return a 400 error. Then, we use OpenAI's Responses API with three key pieces:

- `instructions` that tell the model it's ContractGuard and should decide whether it needs to search the Drive documents.
- A single tool definition named 'supermemory_search', which strictly accepts an object with a `query` string.
- `tool_choice: "auto"`, so the model can either answer directly or invoke our search tool.

At this point, the model returns a sequence of output items, which may include a `function_call` if it thinks a search is required. If there's a function called `supermemory_search` in the output, it means the model wants us to run a document search.

So, we use Supermemory's `.search.execute()` method, we run that `query` against the user's connected Drive docs, limiting the results to the top 5.

Then, we hit the Responses API again, with the original function_call object, the search results, and the user's original message.

Lastly, we have a simple utility to wipe the entire chat history.

```
 const clearChat = () =&gt; {   setMessages([]) }
```

We've also created the UI for the app below and linked the functions to distinct buttons:

```
return (
&lt;div className='flex flex-col h-screen w-screen bg-gray-50'&gt;
{/* Header */}
&lt;header className='bg-white shadow-sm border-b px-4 py-3'&gt;
&lt;div className='max-w-4xl mx-auto flex items-center justify-between'&gt;
&lt;h1 className='text-xl font-semibold text-gray-800 flex items-center gap-2'&gt;
ContractGuard
&lt;/h1&gt;
&lt;div className='flex items-center gap-2'&gt;
&lt;button
onClick={handleConnect}
className='px-3 py-1 text-sm bg-gray-100 hover:bg-gray-200 rounded-md transition-colors'
&gt;
Connect to Google Drive
&lt;/button&gt;

&lt;button
onClick={handleSync}
className='px-3 py-1 text-sm bg-gray-100 hover:bg-gray-200 rounded-md transition-colors'
&gt;
Sync Google Drive
```

```
&lt;/button&gt;
&lt;button
onClick={clearChat}
className='px-3 py-1 text-sm bg-gray-100 hover:bg-gray-200 rounded-md transition-colors'
&gt;
Clear chats
&lt;/button&gt;
&lt;/div&gt;
&lt;/div&gt;
&lt;/header&gt;

{/* Messages */}
&lt;div className='flex-1 overflow-y-auto px-4 py-6'&gt;
&lt;div className='max-w-4xl mx-auto space-y-6'&gt;
{messages.length === 0 &amp;amp;&amp; (
&lt;div className='text-center text-gray-500 py-12'&gt;
&lt;Bot className='w-12 h-12 mx-auto mb-4 text-gray-300' /&gt;
&lt;p className='text-lg'&gt;Start a conversation with the AI&lt;/p&gt;
&lt;p className='text-sm'&gt;Type your message below to get started&lt;/p&gt;
&lt;/div&gt;
)}

{messages.map(message =&gt; (
&lt;div
key={message.id}
className={`flex gap-3 ${
message.role === 'user' ? 'justify-end' : 'justify-start'
}`}
&gt;
{m
... (truncated)
```

This code implements a connect button, sync button, and a chat interface. Your frontend app should change to the following.

Chat with your assistant

To chat with your AI assistant, first, click Connect to Google Drive, you will be redirected to an OAuth URL to complete your authorization process that wil grant Supermemory access to files in your Google Drive, then upload your Contract and Compliance files to your google drive and click Sync Google Drive to bring Supermemory AI up to date with the new files.

Ask a question to your AI via the chat interface, Supermemory will automatically insert context from the contract and compliance files stored in your Google Drive and your previous conversations with the AI into the AI chat, helping your AI give you the correct answers.

Conclusion

And that’s a wrap. You now have a contract compliance chatbot that:

- Connects itself straight to Google Drive without any manual uploads
- Automatically keeps its document “memory” up to date
- Decides when to dig into your contracts and brings back precise, cited answers

By leaning on Supermemory as your memory layer, you never have to build complex ingestion pipelines or worry about stitching context back into your prompts. Supermemory handles the Drive sync, the embedding, and the context injection so you can focus on tweaking your assistant's prompts, refining its compliance checks, and rolling it out to real users.

Head over to supermemory.ai and grab your API key now, it's free.

What Is Context Engineering?

Context engineering is why your AI agent breaks in production.

Your agent works great in demos. Then users actually use it.

They reference something from last week. The model hallucinates because retrieval pulled stale docs. Responses feel generic because there's no user context loaded. You rewrote the prompt

09 Apr 2026

7 min read

Switching Memory Infrastructure

Most teams don't even consider switching memory infrastructure.

And it's not because of cost. It's not because of performance. It's psychology.

We often see companies sticking with a tool that's "fine" even when something better exists. The better tool isn't competing with the old

09 Apr 2026

3 min read

Top Knowledge Graph Solutions for RAG Applications

Vector databases are great at finding similar text. They're genuinely not great at answering 'why does this connect to that', which requires a knowledge graph. We've been deep in this space for a while now, and honestly the market is confusing. Some tools are

09 Apr 2026

7 min read

Product

- API
- Plugins
- Pricing
- Changelog

Resources

- Documentation

- Blog
- Benchmarks

Company

- About
- Careers
- Privacy
- Terms

Connect

© 2026 Supermemory Inc. All rights reserved.

{selectedCustomer.name}

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Customer Support Bot

> Build an intelligent support system that remembers customer history and provides personalized help

Create a customer support system that remembers every interaction, tracks issues across conversations,
and provides personalized support based on customer history and preferences.

## What You'll Build

A customer support bot that:

* **Remembers customer history** across all conversations and channels
* **Tracks ongoing issues** and follows up automatically
* **Provides personalized responses** based on customer tier and preferences
* **Escalates complex issues** to human agents with full context
* **Learns from resolutions** to improve future responses

## Prerequisites

* Node.js 18+ or Python 3.8+
* Supermemory API key
* OpenAI API key
* Customer database or CRM integration
* Basic understanding of customer support workflows

## Implementation

### Step 1: Customer Context Management



```typescript lib/customer-context.ts theme={null}
```
```

```
import { Supermemory } from 'supermemory'

const client = new Supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY!
})

interface Customer {
id: string
email: string
name: string
tier: 'free' | 'pro' | 'enterprise'
joinDate: string
preferences?: Record
}

interface SupportTicket {
id: string
customerId: string
subject: string
status: 'open' | 'pending' | 'resolved' | 'closed'
priority: 'low' | 'medium' | 'high' | 'urgent'
category: string
createdAt: string
updatedAt: string
assignedAgent?: string
}

export class CustomerContextManager {
private getContainerTag(customerId: string): string {
return `customer_${customerId}`
}

async addInteraction(customerId: string, inter
... (truncated)
```

cookbook document-qa

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Document Q&A System

> Build a chatbot that answers questions from your documents with citations and source references

Create a powerful document Q\&A system that can ingest PDFs, text files, and web pages, then answer
questions with accurate citations. Perfect for documentation sites, research databases, or internal
knowledge bases.

## What You'll Build

A document Q\&A system that:
```

```
* **Ingests multiple file types** (PDFs, DOCX, text, URLs)
* **Answers questions accurately** with source citations
* **Provides source references** with page numbers and document titles
* **Handles follow-up questions** with conversation context
* **Supports multiple document collections** for different topics

## Prerequisites

* Node.js 18+ or Python 3.8+
* Supermemory API key
* OpenAI API key
* Basic understanding of file handling

## Implementation

### Step 1: Document Processing System



```typescript lib/document-processor.ts theme={null}
import { Supermemory } from 'supermemory'

const client = new Supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY!
})

interface DocumentUpload {
file: File
collection: string
metadata?: Record
}

export class DocumentProcessor {
async uploadDocument({ file, collection, metadata = {} }: DocumentUpload) {
try {
const formData = new FormData()
formData.append('file', file)
formData.append('containerTags', JSON.stringify([collection]))
formData.append('metadata', JSON.stringify({
originalName: file.name,
fileType: file.type,
uploadedAt: new Date().toISOString(),
...metadata
}))

const response = await fetch('/api/upload-document', {
method: 'POST',
body: formData

... (truncated)
```

cookbook overview
```

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Cookbook

> Complete examples and recipes for building with Supermemory

The Supermemory Cookbook provides complete, production-ready examples that show how to build real
applications with Supermemory. Each recipe includes full implementation details, best practices, and
common patterns.

## Available Recipes



Build an AI assistant that remembers user preferences and context across conversations



Create a chatbot that answers questions from your documents with citations



Build a support system that remembers customer history and provides personalized help



Complete examples using Vercel AI SDK with Supermemory tools



## Coming Soon

We're working on more comprehensive recipes. Have a suggestion? [Let us
know!](mailto:support@supermemory.ai)



Organize and search through research papers and notes



Build a personalized learning system with user isolation



Create an AI assistant for your codebase and documentation



Process and search through meeting recordings and notes



## Getting Help

Can't find what you're looking for?
```

```
* Browse [Search](/search) for specific feature usage
* Check the [AI SDK Examples](/cookbook/ai-sdk-integration) for complete implementations
* Reach out to [support](mailto:support@supermemory.ai) for help

## Contributing Recipes

Have a great Supermemory use case? We'd love to add it to the cookbook!

[Suggest a recipe →](mailto:support@supermemory.ai?subject=Cookbook%20Recipe%20Suggestion)
```

Build Your Own Perplexity in 15 Minutes With Supermemory

Build Your Own Perplexity in 15 Minutes With Supermemory

Research

Pricing

Docs

Blog

Consumer

Plugins

Login

Developer Console

API keys, usage, docs

Personal App

Your memory dashboard

Research

Pricing

Docs

Blog

Consumer

Plugins

Supermemory App

Developer Console

Build Your Own Perplexity in 15 Minutes With Supermemory

Naman Bansal

11 Aug 2025

• 10 min read

Supermemory has a fascinating open-source tool called OpenSearchAI. It's essentially a search assistant similar to Perplexity, but it remembers everything you've searched for and enriches future responses with that memory.

I thought to myself, “This seems cool. But how complicated is it to build something like this?"

So, I challenged myself: Could I recreate a simplified version of this memory-enhanced Perplexity in just 30 mins?

I did, and in this tutorial, I'm going to walk you through exactly how you can do it too. We’ll build our very own mini-clone of OpenSearch using:

- Supermemory's Memory API: A super-simple way to automatically save and retrieve memories without dealing with complicated vector databases or complex integrations.
- Brave Search API: A fast and easy API to pull relevant, real-time web results.
- OpenAI's GPT Model: Just one straightforward call to fuse memories and web search results into clean, easy-to-understand answers.

Honestly, once I saw it working, I couldn’t believe how easy it had been to set up. Let’s dive right in.

Here's the GitHub repo with the full code for you to follow along.

Video

Planning: A Rough Sketch

The actual coding is pretty straightforward for this kind of app, but getting clear on the overall idea and structure helps a ton.

Here's exactly how I thought through this:

How does the original app work?

Supermemory's OpenSearch AI does two main things really well:

- It searches the web and gives clear, concise answers.
- It automatically remembers previous queries, using those memories to give better answers over time.

So, in simple terms, we have two parts: web searching and memory. I wanted my app to do the same.

Picking the Right Tools

Here's my quick thought process on which tools to pick:

Memory: This is the key feature, and luckily, Supermemory already gives me a simple, ready-to-use Memory API. No complicated databases or setup, just a clean, simple SDK to store and retrieve

memories.

Web Search: I needed a reliable search API to pull fresh web results. Brave Search is fast, simple, free, and developer-friendly.

Answer Generation: To create those nice, readable responses combining memories and web results, a single call to OpenAI's API is perfect.

Quick UI Sketch

Here's the rough sketch I quickly drew up to clarify my thinking:

In essence:

- Frontend: Just a simple input for questions and a nice, clean way to show answers, memories, and web results.
- Backend: Takes user questions, interacts with memory, does a quick web search, and uses AI to generate answers.

Here’s what the flowchart looks like:

That’s it! Let’s get into the actual coding now.

Building this app: step-by-step

Prerequisites

First, let’s create a folder and install `npm`. We’ll then install the dependencies.

```
mkdir mini-opensearch
cd mini-opensearch
npm init -y
npm install express cors dotenv axios supermemory openai
```

- express: Our backend API server.
- cors: Makes sure we don't run into annoying CORS issues.
- dotenv: Simple way to handle API keys and configs.
- axios: Easy HTTP requests.
- supermemory: Easy-to-use memory storage and retrieval.
- openai: For generating AI-powered responses.

Create a free supermemory account and get the API key from your dashboard. Also, go to Brave Search API and register for an account there. They have a free plan we’ll use for this tutorial, but you’ll have to enter your card details.

Finally, generate an OpenAI API Key using these steps.

Then, create a `.env` file in your folder and enter the following:

```
SUPERMEMORY_API_KEY=...
OPENAI_API_KEY=...
BRAVE_SEARCH_API_KEY=...
```

Awesome, now inside your folder, create a file `server.js` to set up the Express backend.

Import everything as follows:

```
// server.js
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import axios from 'axios';
import supermemory from 'supermemory';
import OpenAI from 'openai';
```

```
dotenv.config();
const app = express();

// ── MIDDLEWARE──
// Enable CORS for all routes (you can lock this down to specific origins if you like)
app.use(cors());

// Parse JSON + serve frontend
app.use(express.json());
app.use(express.static('public'));
```

```
const memoryClient = new supermemory({
apiKey: process.env.SUPERMEMORY_API_KEY,
});
const openai = new OpenAI({
apiKey: process.env.OPENAI_API_KEY,
});
```

Let's start by tackling the web searches part using Brave's Search API. We'll create a function that will call the Brave API and get web search results for a query. Based on their documentation, write the following code:

```
async function braveSearch(query, count = 5) {
const params = new URLSearchParams({
q: query,
count: count.toString(),
country: 'US',
search_lang: 'en',
extra_snippets: 'true',
text_decorations: 'false',
});
const url = `https://api.search.brave.com/res/v1/web/search?${params}`;

const resp = await axios.get(url, {
headers: {
'Accept': 'application/json',
'Accept-Encoding': 'gzip',
'X-Subscription-Token': process.env.BRAVE_SEARCH_API_KEY,
},
timeout: 5000,
});

if (resp.status !== 200) {
throw new Error(`Brave Search error ${resp.status}: ${resp.statusText}`);
}
```

```
const results = resp.data.web?.results;
if (!Array.isArray(results)) {
throw new Error(`Unexpected Brave response: ${JSON.stringify(resp.data)}`);
}
return results;
}
```

The `extra_snippets` feature returns snippets from the webpage relating to our query, which can be useful while passing to the LLM and text_decorations has been set to false to prevent HTML from being given back.

Then, we append these params to the URL and send a GET request with the API Key in the `X-Subscription-Token` header. Pretty simple!

Now, let's build the main endpoint of our app that will handle incoming user questions. Here's exactly what our endpoint will do:

- Receive a user question from the frontend.
- Save the user's question into Supermemory for future context.
- Retrieve the 3 most relevant past memories related to this question.
- Fetch fresh web search results from Brave.
- Combine memories and web results in a prompt for OpenAI.
- Generate a crisp, concise answer with OpenAI.
- Save this new AI-generated answer back into memory.
- Send back the generated answer, relevant memories, and web results to the frontend.

Start by creating an Express route:

```
app.post('/api/query', async (req, res) =&gt; {
const { query } = req.body; // User's question
if (!query || typeof query !== 'string') {
return res.status(400).json({ error: 'query must be a non-empty string' });
}
});
```

```
await memoryClient.memories.add({ content: query });
```

```
// Retrieve memories with a limit of 3
const searchRes = await memoryClient.search.execute({ q: query, limit: 3 });

// Get the results from the response
const mems = searchRes.results;
```

```
{
"results": [
{
"documentId": "doc_xyz789",
"chunks": [
{
"content": "Machine learning is a subset of artificial intelligence...",
"isRelevant": true,
```

```
"score": 0.85
}
],
"score": 0.95,
"metadata": {
"source": "web",
"category": "technology"
},
"title": "Introduction to Machine Learning"
}
],
"total": 1,
"timing": 123.45
}
```

- Each memory has multiple "chunks" of text, and each chunk has a relevance score.
- Some chunks aren't relevant (isRelevant: false), and we shouldn't include those.
- I only wanted the single most relevant piece of text from each memory to keep things concise and focused for our AI.

So, here’s the logic I went with:

- Step 1: Loop over the top 3 memories Supermemory gave me (already sorted by overall memory score).
- Step 2: For each memory, find only the relevant chunks (isRelevant: true).
- Step 3: Pick the chunk with the highest score (since that's probably the most useful piece).
- Step 4: Create a simple bullet-point list where each bullet combines the memory's title and the best chunk's content.

Here’s how the code looks:

```
//Take the top 3 memories
const top3 = mems.slice(0, 3);

const memoryContext = top3.map(mem =&gt; {
// Filter relevant chunks only
const relevantChunks = mem.chunks.filter(c =&gt; c.isRelevant);

// Pick the most relevant chunk (highest score)
const bestChunk = relevantChunks.reduce((best, c) =&gt; c.score &gt; best.score ? c : best);

// Format as "Title: content"
return `- ${mem.title}: ${bestChunk.content}`;
}).join('\n');
```

```
const webResults = await braveSearch(query, 5);
```

```
const enrichedResults = webResults.map((r, i) =&gt; {

// Collect all available snippets: description + any extras
const allSnips = [r.description, ...(r.extraSnippets || [])];

// Choose the longest one (assuming more detail = more context)
```

```
const bestSnippet = allSnips.reduce((a, b) =&gt;
a.length &gt; b.length ? a : b
, allSnips[0]);

return {
title: r.title,
url: r.url,
snippet: bestSnippet.trim()
};

});

const webSummaries = enrichedResults
.map((r, i) =&gt; `${i+1}. ${r.snippet} — ${r.url}`)
.join('\n\n');
```

And then, it returns an object with the title, URL, and the best snippet. Finally, we format them in a clean list.

Awesome! Now, we just need to pass all the information we've gathered into our LLM. In the `server.js` file, create the prompts:

```
const systemPrompt = `
You are a helpful assistant.
Use the user's past memories plus these fresh web results to craft a concise answer.
Then list the most relevant URLs at the end.
`.trim();

const userPrompt = `
Query: ${query}

Memories:
${memoryContext}

Web Results:
${webSummaries}

Please answer and then list the top links.
`.trim();
```

```
const chat = await openai.chat.completions.create({
model: 'gpt-4o-mini',
messages: [
{ role: 'system', content: systemPrompt },
{ role: 'user', content: userPrompt },
],
temperature: 0.2,
});

const answer = chat.choices[0].message.content;
```

```
// 6) Save AI answer
await memoryClient.memories.add({ content: answer });

// 7) Return everything
res.json({ answer, memories: mems, searchResults: webResults });
```

```
const PORT = process.env.PORT || 3000;
app.listen(PORT, () =&gt; console.log(`Listening on http://localhost:${PORT}`));
```

Frontend

I'm keeping things simple here. So, I won't be using any React or Tailwind. Just plain old HTML, CSS, and JS.

Our goal is to let the user enter a query, send it to the backend, and show the answer, web results, and relevant past memories.

Inside your project folder, create a `public` folder and an `index.html` file inside it.

Here's the basic HTML skeleton:

```
&lt;!DOCTYPE html&gt;
&lt;html lang="en"&gt;
&lt;head&gt;
&lt;meta charset="UTF-8" /&gt;
&lt;meta name="viewport" content="width=device-width, initial-scale=1.0" /&gt;
&lt;title&gt;Mini OpenSearch AI&lt;/title&gt;

&lt;!-- For rendering Markdown responses and sanitizing --&gt;
&lt;script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"&gt;&lt;/script&gt;
&lt;script
src="https://cdn.jsdelivr.net/npm/[email protected]/dist/purify.min.js"&gt;&lt;/script&gt;

&lt;style&gt;
body {
font-family: sans-serif;
max-width: 800px;
margin: 2rem auto;
line-height: 1.6;
}
input {
width: 70%;
padding: 0.6rem;
font-size: 1rem;
}
button {
padding: 0.6rem 1rem;
font-size: 1rem;
margin-left: 0.5rem;
cursor: pointer;
}
h2 { margin-top: 2rem; }
#answer { background: #f6f8fa; padding: 1rem; white-space: pre-wrap; }
ul { list-style: none; padding: 0; }
li { margin-bottom: 1rem; }
a { color: #0366d6; text-decoration: none; font-weight: bold; }
a:hover { text-decoration: underline; }
.snippet { color: #444; margin-top: 0.25rem; }
&lt;/style&gt;
&lt;/head&gt;
&lt;body&gt;
&lt;h1&gt;Mini OpenSearch AI&lt;/h1&gt;
```

```
&lt;div&gt;
&lt;input id="query" placeholder="Ask something..." /&gt;
&lt;button id="search"&gt;Search&lt;/button&gt;
&lt;/div&gt;

&lt;h2&gt;Answer&lt;/h2&gt;
&lt;div id="answer"&gt;Your AI-enhanced answer will appear here.&lt;/div&gt;

&lt;h2&gt;Web Results&lt;/h2&gt;
&lt;ul id="results"&gt;&lt;/ul&gt;

&lt;h2&gt;Relevant Memories&lt;/h2&gt;
&lt;ul id="memories"&gt;&lt;/ul&gt;
```

Here's what it looks like:

Now, let's add the logic when the user presses 'Search'. Here's how:

```
&lt;script&gt;
const queryInput = document.getElementById('query');
const searchButton = document.getElementById('search');
const answerDiv = document.getElementById('answer');
const resultsList = document.getElementById('results');
const memoriesList = document.getElementById('memories');

searchButton.addEventListener('click', async () =&gt; {
const query = queryInput.value.trim();
if (!query) return alert('Please enter a query.');

// Show loading state
answerDiv.textContent = 'Loading...';
resultsList.innerHTML = '';
memoriesList.innerHTML = '';

try {
const res = await fetch('/api/query', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ query })
});
if (!res.ok) throw new Error(`Server error: ${res.statusText}`);
const { answer, searchResults, memories } = await res.json();

const rawHtml = marked.parse(answer);
answerDiv.innerHTML = DOMPurify.sanitize(rawHtml)

// Render Brave search results
for (const r of searchResults) {
const li = document.createElement('li');
const link = document.createElement('a');
link.href = r.url;
link.target = '_blank';
link.rel = 'noopener';
link.textContent = r.title;
li.appendChild(link);

const snippet = document.createElement('div');
```

```
snippet.className = 'snippet';
snippet.textContent = r.description;
li.appendChild(snippet);

resultsList.appendChild(li);
}

const top3 = (memories || [])
.sort((a, b) =&gt; b.score - a.score)
.slice(0, 3);

for (const mem of top3) {
// find the most relevant chunk
const chunk = Array.isArray(mem.chunks)
? mem.chunks.reduce((best, c) =&gt; c.score &gt; best.score ? c : best, mem.chunks[0])
: { content: '' };

const li = document.cre
... (truncated)
```

Meanwhile, the function extracts the user's query from the input box, sends a POST request to our API endpoint, and waits for the response. Then, it converts the Markdown response to normal text and renders the Brave result and the relevant memories.

And, that's it! You've built your very own Perplexity/Supermemory OpenSearch clone. It wasn't that hard, right?

Now, you can fire away and test this! Start the server using:

```
node server.js
```

Conclusion

And that's a wrap. You now have your own mini clone of OpenSearch AI. The whole point of this wasn't to build something massive or production-ready. It was to show how surprisingly little it takes to wire something like this up when you've got the right tools.

Once you have memory as a primitive, you start thinking differently about what apps can do. You stop worrying about sessions and state and all that, because the app can just remember.

If you're curious, go try Supermemory. Sign up, play around, plug it into whatever you're working on. You'll be surprised how far you can get with a single .add() and .search().

Hope this helped. Go build something fun.

What Is Context Engineering?

Context engineering is why your AI agent breaks in production.

Your agent works great in demos. Then users actually use it.

They reference something from last week. The model hallucinates because retrieval pulled stale docs. Responses feel generic because there's no user context loaded. You rewrote the prompt

09 Apr 2026

7 min read

Switching Memory Infrastructure

Most teams don’t even consider switching memory infrastructure.

And it’s not because of cost. It’s not because of performance. It’s psychology.

We often see companies sticking with a tool that’s “fine” even when something better exists. The better tool isn’t competing with the old

09 Apr 2026

3 min read

Top Knowledge Graph Solutions for RAG Applications

Vector databases are great at finding similar text. They're genuinely not great at answering 'why does this connect to that', which requires a knowledge graph. We've been deep in this space for a while now, and honestly the market is confusing. Some tools are

09 Apr 2026

7 min read

Product

- API
- Plugins
- Pricing
- Changelog

Resources

- Documentation
- Blog
- Benchmarks

Company

- About
- Careers
- Privacy
- Terms

Connect

© 2026 Supermemory Inc. All rights reserved.

□ Personal AI Assistant

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Personal AI Assistant

> Build an AI assistant that remembers user preferences, habits, and context across conversations

Build a personal AI assistant that learns and remembers everything about the user - their preferences,
habits, work context, and conversation history.

## What You'll Build

A personal AI assistant that:

* **Remembers user preferences** (dietary restrictions, work schedule, communication style)
* **Maintains context** across multiple chat sessions
* **Provides personalized recommendations** based on user history
* **Handles multiple conversation topics** while maintaining context

## Choose Your Implementation



Thoroughly tested, production-ready. Uses FastAPI + Streamlit + OpenAI.



Modern React approach. Uses Next.js + Vercel AI SDK + Supermemory tools.



## Prerequisites

* **Python 3.8+** or **Node.js 18+**
* **Supermemory API key** ([get one here](https://console.supermemory.ai))
* **OpenAI API key** ([get one here](https://platform.openai.com/api-keys))


Never hardcode API keys in your code. Use environment variables.


***

## Python Implementation

### Step 1: Project Setup

```bash theme={null}
mkdir personal-ai && cd personal-ai
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
pip install supermemory openai fastapi uvicorn python-dotenv streamlit requests
```
```

```
Create a `.env` file:

```bash theme={null}
SUPERMEMORY_API_KEY=your_supermemory_key_here
OPENAI_API_KEY=your_openai_key_here
```

Step 2: Backend (FastAPI)

Create `main.py`. Let's build it step by step:

Import Dependencies

```python theme={null}
from fastapi import FastAPI, HTTPException
from fastapi.responses import StreamingResponse
from openai import AsyncOpenAI
from supermemory import Supermemory
import json
import os
import uu
... (truncated)
```

Model Context Protocol (MCP)

Pages	Content Size
2	7,807 characters

supermemory-mcp mcp

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Overview

> Give your AI assistants persistent memory with the Model Context Protocol

Supermemory MCP Server 4.0 gives AI assistants (Claude, Cursor, Windsurf, etc.) persistent memory across
conversations. Built on Cloudflare Workers with Durable Objects for scalable, persistent connections.

## Quick Install

```bash theme={null}
npx -y install-mcp@latest https://mcp.supermemory.ai/mcp --client claude --oauth=yes
```
```
```

Replace `claude` with your MCP client: `cursor`, `windsurf`, `vscode`, etc.

Manual Configuration

Add to your MCP client config:

```json theme={null}
{
"mcpServers": {
"supermemory": {
"url": "https://mcp.supermemory.ai/mcp"
}
}
}
```

The server uses **OAuth** by default. Your client will discover the authorization server via `/.well-known/oauth-protected-resource` and prompt you to authenticate.

API Key Authentication (Alternative)

If you prefer API keys over OAuth, get one from app.supermemory.ai and pass it in the `Authorization` header:

```json theme={null}
{
"mcpServers": {
"supermemory": {
"url": "https://mcp.supermemory.ai/mcp",
"headers": {
"Authorization": "Bearer sm_your_api_key_here"
}
}
}
}
```

API keys start with `sm_` and skip OAuth when provided.

Project Scoping

Scope all operations to a specific project with `x-sm-project`:

```json theme={null}
{
"mcpServers": {
"supermemory": {
"url": "https://mcp.supermemory.ai/mcp",
"headers": {
"x-sm-project": "your-project-id"
}
}
}
}
```

Tools

```
### `memory`

Save or forget information about the user.

| Parameter | Type | Required | Description |
| -------------- | ---------------------- | -------- | ---------------------
... (truncated)
```

supermemory-mcp setup

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Setup and Usage

> How to set up and use Supermemory MCP Server 4.0

## Quick Install (Recommended)

```bash theme={null}
npx -y install-mcp@latest https://mcp.supermemory.ai/mcp --client claude --oauth=yes
```

Replace `claude` with your MCP client: `cursor`, `windsurf`, `vscode`, etc.

Manual Configuration

Add this to your MCP client config (Claude Desktop, Cursor, Windsurf, etc.):

```json theme={null}
{
"mcpServers": {
"supermemory": {
"url": "https://mcp.supermemory.ai/mcp"
}
}
}
```

The server uses **OAuth authentication** by default. Your MCP client will automatically discover the
authorization server via `/.well-known/oauth-protected-resource` and prompt you to authenticate.

API Key Authentication (Alternative)

If you prefer to use an API key instead of OAuth, get one from
app.supermemory.ai and pass it in the `Authorization` header:

```json theme={null}
{
"mcpServers": {
"supermemory": {
```
```

```
"url": "https://mcp.supermemory.ai/mcp",
"headers": {
"Authorization": "Bearer sm_your_api_key_here"
}
}
}
}
```

API keys start with `sm_` and are automatically detected. When an API key is provided, OAuth authentication is skipped.

Project Scoping (Optional)

To scope all operations to a specific project, add the `x-sm-project` header:

```json theme={null}
{
"mcpServers": {
"supermemory": {
"url": "https://mcp.supermemory.ai/mcp",
"headers": {
"x-sm-project": "your-project-id"
}
}
}
}
```

This keeps memories organized by project, useful when working on multiple codebases or contexts.

Client-Specific Setup

Claude Desktop

For a screenshot-backed walkthrough (Developer → Edit Config, `claude_desktop_config.json`, Connectors), see **[Claude Deskto
... (truncated)

MemoryBench

Pages	Content Size
6	361,674 characters

memorybench architecture

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Architecture

> Understanding MemoryBench's design and implementation

## System Overview

```mermaid theme={null}
flowchart TB
B["Benchmarks(LoCoMo, LongMemEval..)"]
P["Providers(Supermemory, Mem0, Zep)"]
J["Judges(GPT-4o, Claude..)"]

B --> O[Orchestrator]
P --> O
J --> O

O --> Pipeline

subgraph Pipeline[" "]
direction LR
I[Ingest] --> IX[Indexing] --> S[Search] --> A[Answer] --> E[Evaluate]
end

style B fill:#E0F2FE,stroke:#0369A1,color:#0C4A6E
style P fill:#E0F2FE,stroke:#0369A1,color:#0C4A6E
style J fill:#E0F2FE,stroke:#0369A1,color:#0C4A6E
style O fill:#0369A1,stroke:#0369A1,color:#fff
style I fill:#F1F5F9,stroke:#64748B,color:#334155
style IX fill:#F1F5F9,stroke:#64748B,color:#334155
style S fill:#F1F5F9,stroke:#64748B,color:#334155
style A fill:#F1F5F9,stroke:#64748B,color:#334155
style E fill:#F1F5F9,stroke:#64748B,color:#334155
```

Core Components

Component	Role
Benchmarks	Load test data and provide questions with ground truth answers
Providers	Memory services being evaluated (handle ingestion and search)
Judges	LLM-based evaluators that score answers against ground truth

See [Integrations](/memorybench/integrations) for all supported benchmarks, providers, and models.

Pipeline

```mermaid theme={null}
flowchart LR
A[Ingest] --> B[Index] --> C[Search] --> D[Answer] --> E[Evaluate] --> F[Report]

style A fill:#E0F2FE,stroke:#0369A1,color:#0C4A6E
style B fill:#E0F2FE,stroke:#0369A1,color:#0C4A6E
style C fill:#E0F2FE,stroke:#0369A1,color:#0C4A6E
style D fill:#E0F2FE,stroke:
```
```

```
... (truncated)
```

memorybench cli

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# CLI Reference

> Command-line interface for running MemoryBench evaluations

## Commands

### run

Execute the full benchmark pipeline.

```bash theme={null}
bun run src/index.ts run -p -b -j -r
```

Option	Description
`-p, --provider`	Memory provider (`supermemory`, `mem0`, `zep`)
`-b, --benchmark`	Benchmark (`locomo`, `longmemeval`, `convomem`)
`-j, --judge`	Judge model (default: `gpt-4o`)
`-r, --run-id`	Run identifier (auto-generated if omitted)
`-m, --answering-model`	Model for answer generation (default: `gpt-4o`)
`-l, --limit`	Limit number of questions
`-s, --sample`	Sample N questions per category
`--sample-type`	Sampling strategy: `consecutive` (default), `random`
`--force`	Clear checkpoint and restart

See [Supported Models](/memorybench/supported-models) for all available judge and answering models.

***

compare

Run benchmark across multiple providers in parallel.

```bash theme={null}
bun run src/index.ts compare -p supermemory,mem0,zep -b locomo -j gpt-4o
```

***

test

Evaluate a single question for debugging.

```bash theme={null}
```
```

```
bun run src/index.ts test -r -q
```

***

status

Check progress of a run.

```bash theme={null}
bun run src/index.ts status -r
```

***

show-failures

Debug failed questions with full context.

```bash theme={null}
bun run src/index.ts show-failures -r
```

***

list-questions

Browse benchmark questions.

```bash theme={null}

... (truncated)
```

memorybench contributing

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Contributing

> Guidelines for contributing to MemoryBench

## Getting Started

1. Fork the repository
2. Clone your fork:
```bash theme={null}
git clone https://github.com/YOUR_USERNAME/memorybench
cd memorybench
bun install
```
3. Create a branch:
```

```bash theme={null}
git checkout -b feature/your-feature
```

Development Workflow

Running Tests

```bash theme={null}
bun test
```

Running the CLI

```bash theme={null}
bun run src/index.ts
```

Running the Web UI

```bash theme={null}
cd ui
bun run dev
```

Code Structure

Directory	Purpose
`src/cli/`	CLI commands
`src/orchestrator/`	Pipeline execution
`src/benchmarks/`	Benchmark adapters
`src/providers/`	Provider integrations
`src/judges/`	LLM judge implementations
`src/types/`	TypeScript interfaces
`ui/`	Next.js web interface

Contribution Types

Adding a Provider

See [Extending MemoryBench](/memorybench/extend-provider) for the full guide.

1. Create `src/providers/yourprovider/index.ts`
2. Implement the `Provider` interface
3. Register in `src/providers/index.ts`
4. Add config in `src/utils/config.ts`
5. Submit PR with tests

Adding a Benchmark

1. Create `src/benchmarks/yourbenchmark/index.ts`
2. Implement the `Benchmark` interface
3. Register in `src/benchmarks/index.ts`
4. Document question types
5. Submit PR with sample data

```
### Bug Fixes

1. Create an issue describing the bug
2. Reference the issue in your PR
3. Include test cases that reproduce the bug

## Pull Request Guidelines

* Keep PRs focused on a single change
* Update documentation if needed
* Ensure all tes
... (truncated)
```

memorybench extend-benchmark

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Extend Benchmark

> Add a custom benchmark dataset to MemoryBench

## Benchmark Interface

```typescript theme={null}
interface Benchmark {
name: string
load(config?: BenchmarkConfig): Promise
getQuestions(filter?: QuestionFilter): UnifiedQuestion[]
getHaystackSessions(questionId: string): UnifiedSession[]
getGroundTruth(questionId: string): string
getQuestionTypes(): QuestionTypeRegistry
}
```

***

Adding a Custom Benchmark

1. Create the Benchmark

```typescript theme={null}
// src/benchmarks/mybenchmark/index.ts
import type { Benchmark, UnifiedQuestion, UnifiedSession } from "../../types"

export class MyBenchmark implements Benchmark {
name = "mybenchmark"
private questions: UnifiedQuestion[] = []
private sessions: Map = new Map()

async load() {
const data = await this.loadDataset()
```
```

```
this.processData(data)
}

getQuestions(filter?: QuestionFilter) {
let result = [...this.questions]
if (filter?.limit) result = result.slice(0, filter.limit)
return result
}

getHaystackSessions(questionId: string) {
return this.sessions.get(questionId) || []
}

getGroundTruth(questionId: string) {
return this.questions.find(q => q.questionId === questionId)?.groundTruth || ""
}

getQuestionTypes() {
return {
"type1": { id: "type1", description: "Type 1 questions" },
"type2": { id: "type2", description: "Type 2 questions" },
}
}
}
```

2. Register the Benchmark

```typescript theme={null}
// src/benchmarks/index.ts
import { MyBenchmark } from "./mybenchmark"

export const benchmarks = {
locomo: LoComoBenchmark,
longmemeval: LongMemEvalBenchmark,
convomem: ConvoMemBenchmark,
mybenchmark: MyBenchmark, // Add here
}
```
```

# memorybench extend-provider

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

Extend Provider

> Add a custom memory provider to MemoryBench

Provider Interface

```typescript theme={null}
```

```
interface Provider {
name: string
prompts?: ProviderPrompts
initialize(config: ProviderConfig): Promise
ingest(sessions: UnifiedSession[], options: IngestOptions): Promise
awaitIndexing(result: IngestResult, containerTag: string): Promise
search(query: string, options: SearchOptions): Promise
clear(containerTag: string): Promise
}
```

***

Adding a Custom Provider

1. Create the Provider

```typescript theme={null}
// src/providers/myprovider/index.ts
import type { Provider, ProviderConfig, UnifiedSession } from "../../types"

export class MyProvider implements Provider {
name = "myprovider"
private client: MyClient | null = null

async initialize(config: ProviderConfig) {
this.client = new MyClient({ apiKey: config.apiKey })
}

async ingest(sessions: UnifiedSession[], options: IngestOptions) {
const documentIds: string[] = []
for (const session of sessions) {
const response = await this.client.add({
content: JSON.stringify(session.messages),
metadata: session.metadata
})
documentIds.push(response.id)
}
return { documentIds }
}

async awaitIndexing(result: IngestResult) {
// Poll until indexing complete
}

async search(query: string, options: SearchOptions) {
return await this.client.search({ q: query, limit: 10 })
}

async clear(containerTag: string) {
await this.client.delete(containerTag)
}
}
```

2. Register the Provider

```typescript theme={null}
// src/providers/index.ts
import { MyProvider } from "./myprovider"

export const providers = {
supermemory: SupermemoryProvider,
mem0: Mem0Provider,
zep: ZepProvider,
myprovider: MyProvider, // Add her
... (truncated)
```

Search code, repositories, users, issues, pull requests...

GitHub - supermemoryai/memorybench: Unified benchmark for evaluating conversational memory and RAG across multiple datasets · GitHub

Skip to content

Navigation Menu

Toggle navigation

Sign in

Appearance settings

- Platform
- AI CODE CREATION
- GitHub CopilotWrite better code with AI
- GitHub SparkBuild and deploy intelligent apps
- GitHub ModelsManage and compare prompts
- MCP RegistryNewIntegrate external tools
- DEVELOPER WORKFLOWS
- ActionsAutomate any workflow
- CodespacesInstant dev environments
- IssuesPlan and track work
- Code ReviewManage code changes
- APPLICATION SECURITY
- GitHub Advanced SecurityFind and fix vulnerabilities
- Code securitySecure your code as you build
- Secret protectionStop leaks before they start
- EXPLORE

- Why GitHub
- Documentation
- Blog
- Changelog
- Marketplace

View all features

- Solutions
- BY COMPANY SIZE
- Enterprises
- Small and medium teams
- Startups
- Nonprofits
- BY USE CASE
- App Modernization
- DevSecOps
- DevOps
- CI/CD
- View all use cases
- BY INDUSTRY
- Healthcare
- Financial services
- Manufacturing
- Government
- View all industries

View all solutions

- Resources
- EXPLORE BY TOPIC
- AI
- Software Development
- DevOps
- Security
- View all topics
- EXPLORE BY TYPE

- Customer stories
- Events & webinars
- Ebooks & reports
- Business insights
- GitHub Skills
- SUPPORT & SERVICES
- Documentation
- Customer support
- Community forum
- Trust center
- Partners

View all resources

- Open Source
- COMMUNITY
- GitHub SponsorsFund open source developers
- PROGRAMS
- Security Lab
- Maintainer Community
- Accelerator
- GitHub Stars
- Archive Program
- REPOSITORIES
- Topics
- Trending
- Collections
- Enterprise
- ENTERPRISE SOLUTIONS
- Enterprise platformAI-powered developer platform
- AVAILABLE ADD-ONS
- GitHub Advanced SecurityEnterprise-grade security features
- Copilot for BusinessEnterprise-grade AI features
- Premium SupportEnterprise-grade 24/7 support
- Pricing

Search or jump to...

Search code, repositories, users, issues, pull requests...

Search

Clear

Search syntax tips

Provide feedback

We read every piece of feedback, and take your input very seriously.

Include my email address so I can be contacted

Cancel

Submit feedback

Saved searches

Use saved searches to filter your results more quickly

Name

Query

To see all available qualifiers, see our documentation.

Cancel

Create saved search

Sign in

Sign up

Appearance settings

Resetting focus

You signed in with another tab or window. Reload to refresh your session.

You signed out in another tab or window. Reload to refresh your session.

You switched accounts on another tab or window. Reload to refresh your session.

Dismiss alert

{{ message }}

supermemoryai

memorybench

Public

Notifications

You must be signed in to change notification settings

Fork

Star

Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

Additional navigation options

Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

supermemoryai/memorybench

main

18 Branches0 Tags

Go to file

CodeOpen more actions menu

Folders and files

NameNameLast commit message

Last commit date

Latest commit

Dhravya

Merge pull request #33 from supermemoryai/add-memscore

Open commit details

Mar 23, 2026

852c38e · Mar 23, 2026

History

64 CommitsOpen commit details

64 Commits

skills/memorybench

skills/memorybench

Add benchmark-context skill with description

Feb 26, 2026

Merge pull request #33 from supermemoryai/add-memscore

Mar 23, 2026

Fix memscore review issues: cache encoders, truthiness checks, format…

Mar 23, 2026

.gitignore

.gitignore

init: memorybench benchmarking framework

Dec 24, 2025

.prettierignore

.prettierignore

prettier: formatting

Jan 3, 2026

.prettierrc

.prettierrc

prettier: formatting

Jan 3, 2026

LICENSE

LICENSE

Create LICENSE

Dec 24, 2025

README.md

README.md

Add MemScore section to README with link to full docs

Mar 23, 2026

bun.lock

bun.lock

Fix memscore review issues: cache encoders, truthiness checks, format…

Mar 23, 2026

framework.md

framework.md

init: memorybench benchmarking framework

Dec 24, 2025

package.json

package.json

Fix memscore review issues: cache encoders, truthiness checks, format…

Mar 23, 2026

tsconfig.json

tsconfig.json

init: memorybench benchmarking framework

Dec 24, 2025

View all files

Repository files navigation

- README
- MIT license

MemoryBench

A pluggable benchmarking framework for evaluating memory and context systems.

Features

- □ Interoperable: mix and match any provider with any benchmark
- □ Bring your own benchmarks: plug in custom datasets and tasks
- □□ Checkpointed runs: resume from any pipeline stage (ingest → index → search → answer → evaluate)
- □ Multi□provider comparison: run the same benchmark across providers side□by□side

- □ Judge□agnostic: swap GPT□4o, Claude, Gemini, etc. without code changes
- □ Structured reports: export run status, failures, and metrics for analysis
- □□ Web UI: inspect runs, questions, and failures interactively, in real-time!

```
┌──────────────┐ ┌──────────────┐ ┌──────────────┐
│ Benchmarks │ │ Providers │ │ Judges │
│ (LoCoMo, │ │ (Supermem, │ │ (GPT-4o, │
│ LongMem..) │ │ Mem0, Zep) │ │ Claude..) │
└──────┬───────┘ └──────┬───────┘ └──────┬───────┘
└──────────────────┼──────────────────┘
▼
┌──────────────────────┐
│ MemoryBench │
└──────────┬───────────┘
▼
┌────────┬─────────┬────────┬─────────┬────────┐
│ Ingest │ Indexing│ Search │ Answer │Evaluate│
└────────┴─────────┴────────┴─────────┴────────┘
```

Quick Start

```
bun install
cp .env.example .env.local # Add your API keys
bun run src/index.ts run -p supermemory -b locomo
```

Configuration

```
# Providers (at least one)
SUPERMEMORY_API_KEY=
MEM0_API_KEY=
ZEP_API_KEY=

# Judges (at least one)
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
GOOGLE_API_KEY=
```

Commands

Command

Description

`run`

Full pipeline: ingest → index → search → answer → evaluate → report

`compare`

Run benchmark across multiple providers simultaneously

`ingest`

Ingest benchmark data into provider

`search`

Run search phase only

`test`

Test single question

`status`

Check run progress

`list-questions`

Browse benchmark questions

`show-failures`

Debug failed questions

`serve`

Start web UI

`help`

Show help (`help providers`, `help models`, `help benchmarks`)

Options

```
-p, --provider Memory provider (supermemory, mem0, zep)
-b, --benchmark Benchmark (locomo, longmemeval, convomem)
-j, --judge Judge model (gpt-4o, sonnet-4, gemini-2.5-flash, etc.)
-r, --run-id Run identifier (auto-generated if omitted)
-m, --answering-model Model for answer generation (default: gpt-4o)
-l, --limit Limit number of questions
-q, --question-id Specific question (for test command)
--force Clear checkpoint and restart
```

Examples

```
# Full run
bun run src/index.ts run -p mem0 -b locomo

# With custom run ID
bun run src/index.ts run -p mem0 -b locomo -r my-test

# Resume existing run
bun run src/index.ts run -r my-test

# Limited questions
bun run src/index.ts run -p supermemory -b locomo -l 10

# Different models
bun run src/index.ts run -p zep -b longmemeval -j sonnet-4 -m gemini-2.5-flash

# Compare multiple providers
bun run src/index.ts compare -p supermemory,mem0,zep -b locomo -s 5

# Test single question
bun run src/index.ts test -r my-test -q question_42
```

```
# Debug
bun run src/index.ts status -r my-test
bun run src/index.ts show-failures -r my-test
```

Pipeline

```
1. INGEST Load benchmark sessions → Push to provider
2. INDEX Wait for provider indexing
3. SEARCH Query provider → Retrieve context
4. ANSWER Build prompt → Generate answer via LLM
5. EVALUATE Compare to ground truth → Score via judge
6. REPORT Aggregate scores → Output accuracy + latency
```

Each phase checkpoints independently. Failed runs resume from last successful point.

MemScore

MemScore is a composite metric that captures three dimensions of provider performance in a single line:

```
accuracy% / latencyMs / contextTokens
```

Component

What it measures

Quality

Answer accuracy — `(correct / total) * 100` from judge evaluations

Latency

Average search response time in milliseconds

Tokens

Average context tokens sent to the answering model (counted client-side)

After a run completes, MemScore appears in the CLI summary:

```
Summary:
Total Questions: 50
Correct: 43
Accuracy: 86.00%
MemScore: 86% / 145ms / 1823tok
```

MemScore is intentionally a triple, not a single number — collapsing quality, latency, and cost into one score hides important tradeoffs. Use it to compare providers side-by-side on the same benchmark:

```
bun run src/index.ts compare -p supermemory,mem0,zep -b locomo -j gpt-4o
```

The `report.json` includes both a display string and structured `memscoreComponents` for programmatic use.

Full MemScore documentation →

Checkpointing

Runs persist to `data/runs/{runId}/`:

- `checkpoint.json` - Run state and progress
- `results/` - Search results per question
- `report.json` - Final report

Re-running same ID resumes. Use `--force` to restart.

Extending

Component

Guide

Add Provider

src/providers/README.md

Add Benchmark

src/benchmarks/README.md

Add Judge

src/judges/README.md

Project Structure

src/README.md

License

About

Unified benchmark for evaluating conversational memory and RAG across multiple datasets

supermemory.ai/docs/memorybench/overview

Topics

benchmark-framework

ai-memory

Resources

Readme

License

MIT license

Uh oh!

There was an error while loading. Please reload this page.

Activity

Custom properties

Stars

stars

Watchers

watching

Forks

forks

Report repository

Uh oh!

There was an error while loading. Please reload this page.

Contributors

Languages

TypeScript

99.2%

Other

0.8%

Footer

© 2026 GitHub, Inc.

Footer navigation

Terms

Privacy

Security

Status

Community

Docs

Contact

Manage cookies

Do not share my personal information

You can’t perform that action at this time.

Migration

Pages	Content Size
1	9,907 characters

migration from-zep

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Migrating from Zep to Supermemory

> Quick guide to migrate from Zep to Supermemory

## Key Differences

| Zep AI | Supermemory |
| ---------------------------------------- | ---------------------------------------------- |
| Sessions & Messages | Documents & Container Tags |
| `session.create()` | Use `containerTags` parameter |
| `memory.add(session_id, ...)` | `add({containerTag: [...]})` |
| `memory.search(session_id, {text: ...})` | `search.execute({q: ..., containerTags: [...]})` |

## Installation


```bash Python theme={null}
pip install supermemory
```

```bash TypeScript theme={null}
npm install supermemory
```



```python Python theme={null}
from supermemory import Supermemory
client = Supermemory(api_key="your-api-key")
```

```typescript TypeScript theme={null}
import { Supermemory } from "supermemory";
const client = new Supermemory({ apiKey: "your-api-key" });
```


API Mapping

Session Management
```

```python Zep AI theme={null}
session = client.session.create(
session_id="user_123",
user_id="user_123"
)
```

```python Supermemory theme={null}
# No explicit session creation - use containerTag
containerTag = ["user_123"]
```

Adding Memories

```python Zep AI theme={null}
client.memory.add(
session_id="user_123",
memory={"content": "User prefers dark mode"}
)
```

```python Supermemory theme={null}
client.add({
"content": "User prefers dark mode",
"containerTag": ["user_123"]
})
```

Searching

```python Zep AI theme={null}
results = client.memory.search(
session_
... (truncated)
```

Changelog

Pages	Content Size
1	23,501 characters

changelog overview

```
> ## Documentation Index
> Fetch the complete documentation index at: https://supermemory.ai/docs/llms.txt
> Use this file to discover all available pages before exploring further.

# Changelog

> New updates and improvements to Supermemory


### Supermemory CLI

New command-line tool for managing memories, documents, profiles, tags, connectors, and API keys directly
from the terminal.

### `@supermemory/tools` v1.4.1

Now exposes raw `searchResults` in `MemoryPromptData`, giving full control over how retrieved memories
are formatted in prompts.

### PPTX & Audio Ingestion

PowerPoint files (`.pptx`) are now a supported content type. Audio files are automatically transcribed via
Gemini 2.5 Flash, chunked, and indexed.

### Multi-containerTag Scoped API Keys

Scoped API keys can now be assigned to multiple container tags — one key, multiple spaces.

### Console: Documents Page

New dedicated documents browser in the console for viewing, filtering, and managing all ingested content.



### Delete Scoped API Keys

New `DELETE` endpoint to disable scoped API keys programmatically.

### `supermemory-agent-framework` Python Package

Official Python package for using Supermemory with Microsoft's Agent Framework — memory tools and
middleware out of the box.

### Interactive Memory Graph in MCP

The MCP server now includes an interactive graph visualization app for exploring memory connections from
any MCP-compatible client.

### More Integrations

* **ViaSocket** — new integration guide for automation workflows.
* **Plugin Auth Connect Page** — OAuth-style connect page for Claude Code, OpenCode, and OpenClaw.
* **OpenAI SDK Backfill** — improved compatibility across TypeScript and Python SDKs.

### Other

* **Retry failed documents** by re-submitting with the same `customId`.
```

```
* **Delete connection without documents** — disconnect a source without deleting synced content.
* **Org-level overage toggle** in billing settings.
*
... (truncated)
```